



COMPLETE LEARNING FLOW

React.js

Developer Notes

BEGINNER → ADVANCED → PROFESSIONAL

TABLE OF CONTENTS (I)

01 Introduction to React

- What is React?
- Why React?
- Features
- Virtual DOM
- Real DOM vs Virtual DOM
- SPA (Single Page Application)
- React Ecosystem
- React Developer Roadmap

02 Environment Setup

- Node.js & npm / npx
- Vite vs Create React App (CRA)
- Project Structure
- Running React Application

03 JSX (JavaScript XML)

- What is JSX? & Syntax
- Expressions & Attributes
- Fragments & Comments
- JSX Rules

04 Components

- Functional Components
- Class Components (Overview)
- Component Structure & Nested
- Reusable Components
- Component Best Practices

05 Props

- Passing & Receiving Props
- Default & Children Props
- Prop Drilling
- Prop Validation

06 State Management

- useState() & Updating State
- State vs Props
- Object State & Array State
- State Best Practices

07 Event Handling

- Click, Form, Keyboard, Mouse Events
- Event Object
- Passing Arguments

08 Conditional Rendering

- if Statements & Ternary Operator
- Logical &&
- Switch & Multiple Conditions

09 Lists & Keys

- map() & Rendering Lists
- Keys & Dynamic Lists
- Best Practices

10 Forms

- Controlled Components
- Uncontrolled Components
- Input Fields (Textarea, Select, etc.)
- Form Submission

11 React Hooks

- useState() & useEffect()
- useRef() & useContext()
- useMemo() & useCallback()
- useReducer() & useLayoutEffect()
- Custom Hooks

12 Component Lifecycle

- Mounting & Updating
- Unmounting & Cleanup Function
- Lifecycle with Hooks

13 Styling React

- CSS, CSS Modules, Inline CSS
- Tailwind CSS & Bootstrap
- Styled Components & Sass

TABLE OF CONTENTS (II)

14 Routing

- React Router & BrowserRouter
- Routes, Route, Link, NavLink
- Nested & Dynamic Routes
- URL Parameters & Protected Routes

15 API Integration

- Fetch API & Axios
- GET, POST, PUT, DELETE
- Loading State & Error Handling
- Async/Await

16 Context API

- createContext() & Provider / Consumer
- useContext()
- Global State & Theme Management

17 Advanced State Mgmt

- Redux Toolkit & Redux Store
- Slice, Dispatch, Selector
- Middleware & Redux DevTools
- Zustand (Overview)

18 Performance Optimization

- React.memo()
- useMemo() & useCallback()
- Lazy Loading & Suspense
- Code Splitting & Virtualization

19 Error Handling

- Error Boundaries
- Try-Catch & API Errors
- Validation & Debugging

20 Testing

- React Testing Library & Jest
- Unit & Component Testing
- Mock Functions & Integration Testing

21 Authentication

- JWT Authentication
- Login, Logout, Protected Routes
- Refresh Token & Session Mgmt

22 React Developer Tools

- React DevTools & Chrome DevTools
- ESLint & Prettier
- Vite Dev Server

23 React + Backend

- REST APIs
- Node.js & Express.js
- MongoDB, Firebase, Supabase
- Authentication APIs

24 Next.js (Framework)

- Introduction & File-based Routing
- Server vs Client Components
- API Routes
- SSR, SSG & Deployment

25 Deployment

- Vercel, Netlify, Render
- GitHub Pages
- Environment Variables
- Production Build

26 Real-World Projects

- Counter, Calculator, Weather App
- Todo, Notes, Expense Tracker
- Movie Search, Blog, Chat App
- E-commerce, Admin Dashboard
- Social Media Clone, Food App
- LMS & Full Stack MERN Project

RECOMMENDED LEARNING PATH

L1 Foundation

JavaScript (ES6+) · Intro to React · JSX · Components · Props · State

L2 Core React

Event Handling · Conditional Rendering · Lists & Keys · Forms · Hooks · Component Lifecycle

L3 Intermediate

Styling · Routing · API Integration · Context API · Authentication

L4 Advanced React

Redux Toolkit · Performance Optimization · Error Handling · Testing · React Developer Tools

L5 Professional

Next.js · Backend Integration · Deployment · Production Projects · Open Source · TypeScript · Interview Prep

CAREER PATH

THE JOURNEY

HTML



CSS



JavaScript (ES6+)



Git & GitHub



React.js



React Router



REST APIs



Context API



Redux Toolkit



Next.js



TypeScript



Testing (Jest + RTL)



Node.js + Express.js



MongoDB / SQL



Auth (JWT, OAuth)



Deployment



MERN Stack Developer

01 INTRODUCTION TO REACT

● What is React?

React is an open-source, front-end **JavaScript library** used for building user interfaces (UIs) or UI components. It is maintained by **Meta (formerly Facebook)** and a vast community of individual developers and companies.

Library vs. Framework

A common interview question! React is a **library**, not a framework. A framework (like Angular) dictates how you structure your entire application and includes routing, HTTP clients, etc. React only cares about one thing: *rendering UI components to the screen*. You have the freedom to choose your own tools for routing and state management.

✓ Why Use React?

1. Component Reusability

Break complex UIs into small, isolated pieces of code (components). Build once, reuse everywhere.

2. High Performance

Thanks to the Virtual DOM, React minimizes expensive operations on the actual browser DOM.

3. Massive Community

Millions of developers mean infinite open-source packages, tutorials, and quick solutions to bugs.

4. Learn Once, Write Anywhere

The concepts learned in React.js easily translate to building mobile apps using **React Native**.

Core Features of React

- **JSX (JavaScript XML):** A syntax extension that allows you to write HTML directly inside JavaScript files. It makes writing React components much easier and more intuitive.
- **Unidirectional Data Flow:** In React, data only flows in one direction (top to bottom, from parent to child). This makes the application highly predictable and easier to debug.
- **Declarative UI:** You tell React *what* you want the UI to look like based on the current state, and React figures out *how* to update the DOM to match it.
- **Hooks:** Introduced in React 16.8, hooks (like `useState` and `useEffect`) allow you to use state and lifecycle features in functional components without writing class components.

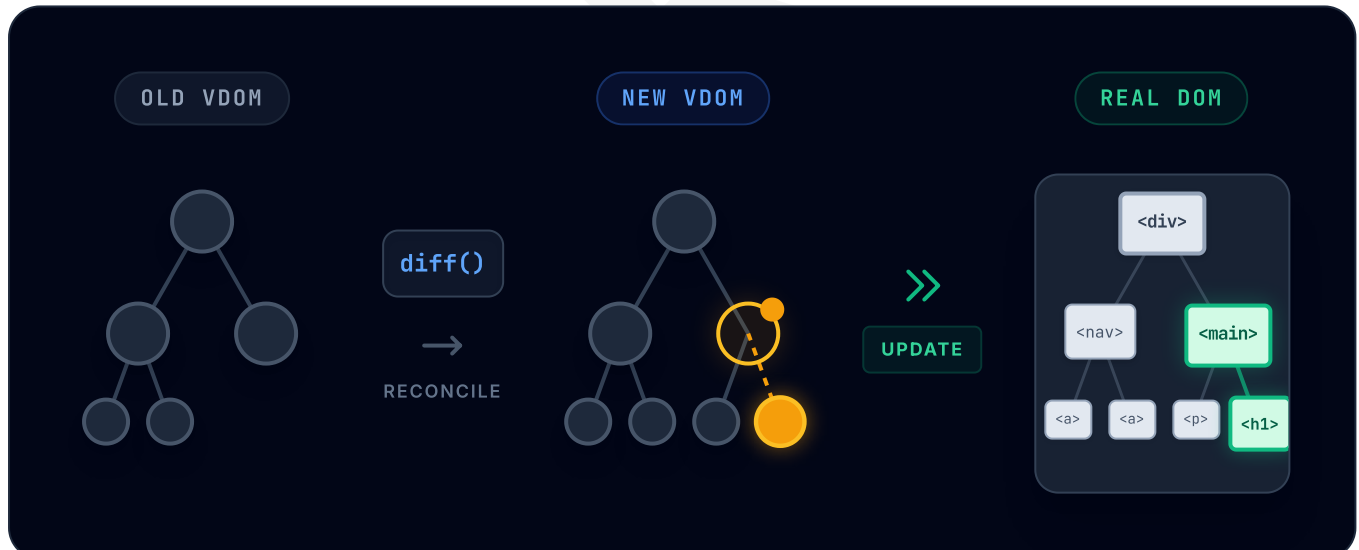
THE VIRTUAL DOM

The Browser DOM (Document Object Model) is a tree-like representation of a web page. However, updating the Real DOM is incredibly slow and expensive in terms of performance. React solves this using the **Virtual DOM**.

What is the Virtual DOM?

It is a lightweight, purely in-memory representation of the Real DOM. It is essentially a giant JavaScript object. When the state of a React component changes, React does not immediately touch the Real DOM. Instead, it updates the Virtual DOM first.

THE RECONCILIATION PROCESS (DIFFING)



React calculates the absolute minimum number of changes required (Diffing) and applies them in a batch (Reconciliation).

REAL DOM VS VIRTUAL DOM

Real DOM	Virtual DOM (React)
Updates are slow and heavy .	Updates are extremely fast and lightweight .
Can directly manipulate HTML elements on the screen.	Cannot directly manipulate HTML; it acts as a middleman.
If an element updates, it forces the entire parent chain to re-render (Repaint).	Updates only the specific node that changed, preventing unnecessary repaints.
High memory wastage in complex applications.	Efficient memory usage due to batch updates.

Interview Tip: If asked "Is the Virtual DOM faster than the Real DOM?", clarify that the Virtual DOM itself isn't what is displayed. It is a *strategy* to make updating the Real DOM faster by minimizing direct manipulations.

ARCHITECTURE & ECOSYSTEM

Single Page Application (SPA)

React is primarily used to build **Single Page Applications**.

Traditional Website (MPA)

Every time you click a link, the browser makes a request to the server. The server generates a brand new HTML page and sends it back. The browser screen goes blank momentarily while the new page loads.

React Application (SPA)

The browser loads a single HTML file (`index.html`) exactly once. When you click a link, React intercepts it, fetches only the required JSON data, and dynamically replaces the UI components on the screen. No page reload occurs.

The React Ecosystem

Because React is a "Library" and not a complete framework, it relies on a massive ecosystem of third-party packages to build a fully functional application.

ROUTING

React Router DOM

Since React is a SPA, it doesn't have multiple pages. React Router fakes the URL changes and determines which component to show based on the URL path.

STATE

Redux Toolkit / Zustand

While React has built-in state (`useState`), managing state across hundreds of components requires external global state management libraries.

FRAMEWORK

Next.js

The industry standard React framework. It sits on top of React and provides Server-Side Rendering (SSR) for better SEO, built-in routing, and full-stack capabilities.



React Developer Roadmap

Please refer to **Page 03** of this document for a complete visual guide mapping the journey from Beginner concepts to Full-Stack MERN Professional.

02 ENVIRONMENT SETUP

Before writing any React code, you must configure your local development environment. React isn't just a simple script you include in an HTML file anymore; it relies on modern build tools to compile JSX, manage dependencies, and optimize code for production.

Node.js

Node.js is a cross-platform, open-source JavaScript runtime environment that allows JavaScript to run on a server (outside the browser).

Interview Question: "Why do we need Node.js for React, which is a frontend library?"

We don't need Node.js to *run* React in production. We need it for the **development environment**. Node.js powers the local development server (like Vite or Webpack), transpiles our JSX into plain JavaScript (using Babel), and manages our project's packages.

npm vs. npx

When you install Node.js, two incredibly powerful CLI (Command Line Interface) tools are installed automatically alongside it: npm and npx. Understanding the difference is crucial.

NPM (NODE PACKAGE MANAGER)

npm is the default package manager for Node.js. It is used to **install, update, and manage dependencies** (third-party libraries) in your project.

```
$ npm install react-router-dom
# Installs the package permanently into
node_modules
```

NPX (NODE PACKAGE EXECUTE)

npx is a package runner. It allows you to **execute CLI tools and binaries** without having to install them globally on your machine.

```
$ npx create-react-app my-app
# Downloads, runs, and deletes the installer
```

Key Rule of Thumb

Use **npm** when you want to add a package to your project for permanent use. Use **npx** when you want to run a one-off command (like bootstrapping a new project) without cluttering your global hard drive space.

BOOTSTRAPPING A PROJECT

Legacy Create React App (CRA)

For years, `npx create-react-app` was the official way to start a React project. It used Webpack under the hood. However, it is now extremely slow, bloated, and officially **deprecated** by the React documentation. Do not use this for modern projects.

Modern Vite (Recommended)

Vite (French for "fast") is a next-generation frontend tooling built by Evan You. It leverages native ES modules in the browser and uses `esbuild` (written in Go), making local server startup and Hot Module Replacement (HMR) insanely fast.

Running the Application (Vite Workflow)

```
bash — codermind@macbook

# 1. Initialize a new Vite project
npm create vite@latest my-react-app -- --template react

# 2. Navigate into the project folder
cd my-react-app

# 3. Install all dependencies (from package.json)
npm install

# 4. Start the development server
npm run dev

VITE v5.0.0 ready in 250 ms
→ Local: http://localhost:5173/
```

Once the server starts, you can open `http://localhost:5173` in your browser. Thanks to **Hot Module Replacement (HMR)**, any time you save a file in your code editor, the browser will instantly update the UI without doing a full page refresh.

PROJECT ANATOMY

Understanding the folder structure is crucial. Below is the anatomy of a standard React project generated by Vite.

my-react-app/

- node_modules/
- public/
 - vite.svg
- src/
 - App.css
 - App.jsx
 - index.css
 - main.jsx
- index.html
- package.json
- vite.config.js

index.html

The **only** HTML file in our Single Page Application. It contains a single empty div: `<div id="root"></div>`. React will inject all UI components into this div.

src/main.jsx

The entry point of the React application. This file grabs the `#root` div from the HTML and renders the main `<App />` component inside it using `ReactDOM.createRoot()`.

src/App.jsx

The root component. Think of it as the main container for your entire application. All other components (like Navbar, Sidebar, Content) will be nested inside this file.

package.json

The heart of the Node project. It keeps a record of all the dependencies (packages) installed via npm, and contains the scripts used to start and build the project.

node_modules/

A massive folder generated automatically by `npm install`. It contains the actual code for all downloaded dependencies. **Never commit this folder to GitHub.**

03 JSX (JAVASCRIPT XML)

What is JSX?

JSX is a syntax extension for JavaScript. It allows you to write HTML-like markup directly inside a JavaScript file. While it is not strictly required to use React, almost all React developers use it because it makes writing UI components incredibly intuitive and readable.

Under the hood (Babel Transpilation)

JSX (WHAT YOU WRITE)

```
const element = <h1 className="title">
  Hello World!
</h1>;
```

JS (WHAT THE BROWSER SEES)

```
const element = React.createElement(
  'h1',
  { className: 'title' },
  'Hello World!'
);
```

The 3 Golden Rules of JSX

1. Return a Single Root Element

A component cannot return multiple parallel elements. You must wrap them in a single parent `<div>` or a Fragment (`<></>`).

```
// ❌ ERROR
return (
  <h1>Hi</h1>
  <p>There</p>
);
```

```
// ✅ CORRECT
return (
  <div>
    <h1>Hi</h1>
    <p>There</p>
  </div>
);
```

2. Close All Tags

JSX requires explicit closing tags. HTML elements that are usually void (like `<img>` or `<input>`) MUST be self-closed.

```
// ❌ ERROR:  <br>
// ✅ CORRECT:  <br />
```

3. camelCase All Attributes

Because JSX turns into JavaScript, attributes written in HTML must be converted to camelCase (e.g., `onClick` becomes `onClick`).

```
// ❌ ERROR: <button tabIndex="1" onclick="submit()">
// ✅ CORRECT: <button tabIndex="1" onClick={submit}>
```

EXPRESSIONS & ATTRIBUTES

JavaScript Expressions { }

The true power of JSX lies in its ability to seamlessly inject dynamic JavaScript data into the HTML structure. To do this, you use **curly braces { }**. Any valid JavaScript *expression* can go inside these braces.

```
const name = "Alice";
const score = 95;

return (
  <div>
    <!-- Displaying Variables -->
    <h1>Welcome, {name}!</h1>

    <!-- Math and Logic -->
    <p>Score Percentage: {score / 100}%</p>

    <!-- Calling Functions -->
    <p>Name in Caps: {name.toUpperCase()}</p>
  </div>
);
```

Warning: Expressions vs Statements. You can only put *expressions* (things that evaluate to a value) inside {}. You CANNOT put statements like `if ... else`, for loops, or variable declarations inside JSX.

Attributes & Inline Styling

You can assign strings to attributes using quotes, just like HTML. But to dynamically assign an attribute, use curly braces *without* quotes.

HTML Syntax	React JSX Syntax	Reason
<code>class="btn"</code>	<code>className="btn"</code>	<code>class</code> is a reserved keyword in JS.
<code>for="email"</code>	<code>htmlFor="email"</code>	<code>for</code> is a reserved keyword in JS (loops).
<code>src="url"</code>	<code>src={dynamicUrl}</code>	No quotes when using dynamic variables.

Inline Styles in JSX

Inline styles are not passed as a string. They must be passed as a **JavaScript Object**. Because of this, it requires double curly braces: `{{ }}`. The outer braces inject JS, and the inner braces create the object. CSS properties become camelCase.

```
// ❌ HTML Style (Will throw an error)
<h1 style="color: red; font-size: 20px">Oops</h1>

// ✅ React JSX Style
<h1 style={{ color: 'red', fontSize: '20px' }}>Success</h1>
```

FRAGMENTS & COMMENTS

React Fragments

Because of the "Single Root Element" rule, developers used to wrap every component in a `<div>`. In a large application, this results in hundreds of useless wrapper `divs`, often referred to as **"Div Soup"**, which can break CSS Grid/Flexbox layouts and slow down rendering.

React Fragments allow you to group a list of children without adding an extra node to the actual browser DOM.

BAD: EXTRA DIV <pre> return (<div> <h1>Title</h1> <p>Desc</p> </div>); </pre> Resulting DOM: <pre> <div id="root"> <div> <!-- Pointless Wrapper --> <h1>Title</h1> <p>Desc</p> </div> </div> </pre>	GOOD: FRAGMENT <pre> return ( <h1>Title</h1> <p>Desc</p> ); </pre> Resulting DOM: <pre> <div id="root"> <h1>Title</h1> <p>Desc</p> </div> </pre>
--	---

Comments in JSX

You cannot use standard HTML comments `<!-- -->` inside a JSX block. Because JSX is JavaScript, you must use standard JavaScript block comments `/* */`, but they **must be wrapped in curly braces {}** so the compiler knows you are evaluating an expression.

```

const Header = () => {
  // Standard JS comments work outside the return statement
  return (
    <div>
      {/* This is a correct JSX comment */}
      <h1>Navigation</h1>
    </div>
  );
};

```

 **Pro Tip for VS Code:** You don't need to manually type the JSX comment syntax. Simply place your cursor on the line inside your JSX and press `Ctrl + /` (Windows/Linux) or `Cmd + /` (Mac). VS Code will automatically wrap it perfectly.

04 COMPONENTS

Components are the core building blocks of any React application. Think of them as custom, reusable HTML elements. A component takes in parameters (Props), manages its own internal data (State), and returns a React element (JSX) describing what should appear on the screen.

Functional vs Class Components

LEGACY Class Component

Prior to React 16.8 (2019), this was the only way to use State and Lifecycle methods. It requires extending `React.Component` and writing a `render()` method. They are now considered legacy.

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hi!</h1>;  
  }  
}
```

MODERN Functional Component

The modern standard. A simple JavaScript function that accepts props and returns JSX. Thanks to **React Hooks**, functional components can now do everything class components could do, but with much cleaner code.

```
const Welcome = () => {  
  return <h1>Hi!</h1>;  
};  
  
export default Welcome;
```

The Component Golden Rule

Component names **MUST start with a capital letter** (e.g., `<Navbar />`, not `<navbar />`). React treats lowercase tags as standard DOM elements (like `div` or `span`). If you use a lowercase letter for a custom component, React will ignore it and render nothing, often without throwing a clear error.

05 PROPS

What are Props?

Props (short for "Properties") are the mechanism React uses to pass data from one component to another. You can think of them as the arguments you pass into a standard JavaScript function, or the attributes you pass into an HTML tag.

- **One-Way Data Flow:** Props can only be passed *down* the component tree (from a Parent component to a Child component). They cannot be passed upwards or sideways.
- **Props are Read-Only (Immutable):** A child component must **never** attempt to modify the props it receives from its parent.

Passing & Receiving Props

1. Parent Component (App.jsx)

```
import UserCard from './UserCard';

const App = () => {
  return (
    <div>
      {/* Passing props like HTML attributes */}
      <UserCard
        name="Alice"
        age={25}
        isPremium={true}
      />
    </div>
  );
};
```

2. Child Component (UserCard.jsx)

```
// Props are received as a single Object
const UserCard = (props) => {
  return (
    <div className="card">
      <h2>Name: {props.name}</h2>
      <p>Age: {props.age}</p>
      {props.isPremium && <span>PRO</span>}
    </div>
  );
};
```

The Modern Approach: Destructuring Props

Writing `props.` over and over is tedious. Modern React developers almost exclusively use JavaScript **Object Destructuring** directly in the function parameter list to extract the exact props they need.

```
// Destructuring 'name' and 'age' directly from the props object
const UserCard = ({ name, age, isPremium }) => {
  return <h2>Hello, {name} ({age})</h2>;
};
```

CHILDREN & VALIDATION

The children Prop

Sometimes you don't know what content a component will hold ahead of time. React provides a special, built-in prop called `children`. It allows you to pass JSX *inside* the opening and closing tags of a custom component, creating layout wrappers.

```
// 1. Parent using the wrapper
return (
  <CardWrapper>
    {/* This UI becomes 'children' */}
    <h1>User Profile</h1>
    <p>Alice in Wonderland</p>
    <button>Edit</button>
  </CardWrapper>
);
```

```
// 2. The Wrapper Component
const CardWrapper = ({ children }) => {
  return (
    <div className="shadow rounded p-4">
      {/* Rendering whatever was passed */}
      {children}
    </div>
  );
};
```

Default Props

If a parent forgets to pass a prop, it will be undefined. To prevent UI crashes, you can assign default values. While React used to rely on a specific `defaultProps` object, the modern standard is to use standard ES6 default parameters during destructuring.

```
// If 'theme' is not provided, it defaults to "light"
const Button = ({ text, theme = "light" }) => {
  return <button className={`btn-${theme}`}>{text}</button>;
};
```

Prop Validation & TypeScript

JavaScript does not enforce data types. If a component expects an Array but receives a String, the app will crash at runtime. Historically, React developers used a library called **PropTypes** to validate data types during development.

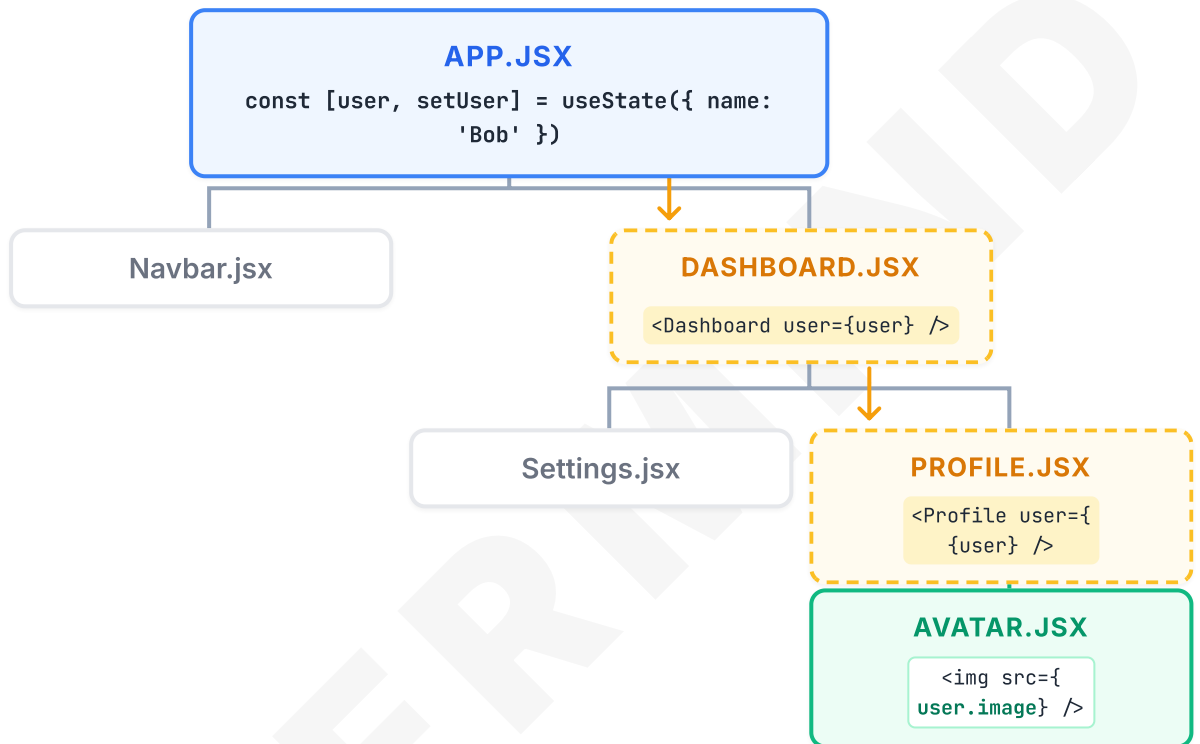
```
// Legacy PropTypes (Deprecated standard)
UserCard.propTypes = {
  age: PropTypes.number.isRequired
};
```

Industry Standard Shift (2024+):

Today, `PropTypes` are rarely used in new projects. The industry standard has entirely shifted to using **TypeScript** to define strict, compile-time interfaces for React Props. We will cover this in Level 5 of the roadmap.

THE PROP DRILLING PROBLEM

Because React utilizes a strict one-way data flow, data must be passed down the component tree manually. **Prop Drilling** occurs when you need to pass data from a high-level component down to a deeply nested component, forcing you to pass the prop through intermediate components that don't actually need the data themselves.



Why is this a problem?

In the diagram above, the Avatar component needs the user data to display an image. However, to get the data from App down to Avatar, it must be passed as a prop through Dashboard and Profile.

Neither Dashboard nor Profile care about the user data, they are just acting as delivery couriers. In a large application with 15+ nested layers, Prop Drilling creates incredibly messy, tightly-coupled code that is difficult to refactor.

06

STATE MANAGEMENT

While Props allow parents to pass data *down* to children, **State** allows a component to manage its own internal data. When a component's state changes, React automatically re-renders that component (and its children) to reflect the new data on the screen.

State vs. Props

Props (Properties)	State
Passed from Parent to Child.	Managed internally within the Component.
Immutable (Read-Only). A component cannot change its own props.	Mutable. A component can update its own state.
Used to configure a component.	Used for dynamic data (user input, fetching data).

The useState() Hook

To add state to a functional component, we use the useState hook. It returns an array with exactly two items: the current state value, and a function (the "setter") used to update that state. We use array destructuring to assign them to variables.

```
import { useState } from 'react';

const Counter = () => {
  // Destructuring: [currentValue, updaterFunction] = useState(initialValue)
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
};
```

Crucial Concept: State Updates are Asynchronous and Batched

When you call `setCount(count + 1)`, React does **not** update the count variable immediately. Instead, it schedules an update and re-renders the component later. Furthermore, if you call multiple setters in the same event handler, React will "batch" them together into a single re-render for performance.

FUNCTIONAL UPDATES & OBJECT STATE

Functional State Updates

Because state updates are asynchronous, if the new state depends on the *previous* state (like incrementing a counter), you should pass a callback function to the setter. React guarantees this callback will receive the most up-to-date state.

```
// ❌ RISKY: Might use a "stale" closure value if updated rapidly
setCount(count + 1);

// ✅ SAFE: Always guarantees the latest previous state
setCount((prevCount) => prevCount + 1);
```

Managing Object State

In React, state should be treated as strictly **immutable**. You must never modify the state object directly (e.g., `user.name = "Alice"`). Instead, you must create a *new* object, copy the existing properties over (using the spread operator `...`), and then overwrite the specific properties you want to change.

❌ THE WRONG WAY (MUTATION)

```
// React won't detect this change!
// The memory address is the same.
const updateUser = () => {
  user.firstName = "Alice";
  setUser(user);
};
```

✅ THE RIGHT WAY (IMMUTABILITY)

```
// Create a brand new object
const updateUser = () => {
  setUser({
    ...user, // Copy existing properties
    firstName: "Alice" // Overwrite
  });
};
```

Handling Form Inputs with Object State

A common pattern is using a single state object to handle multiple inputs in a form by leveraging the input's name attribute.

```
const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData(prev => ({ ...prev, [name]: value }));
};
```

MANAGING ARRAY STATE

Just like objects, Arrays in React state must be treated as immutable. You cannot use methods that mutate the original array, such as `push()`, `pop()`, `splice()`, or `sort()` directly on the state variable. Instead, you must return a new array using methods like the spread operator, `map()`, or `filter()`.

Action	❌ Avoid (Mutates)	✅ Prefer (Returns new array)
Add item	<code>items.push(newItem)</code>	<code>[...items, newItem]</code>
Remove item	<code>items.splice(index, 1)</code>	<code>items.filter(item => item.id !== id)</code>
Update item	<code>items[i] = updatedItem</code>	<code>items.map(item => item.id === id ? updatedItem : item)</code>

Example: A Simple Todo List

```
const TodoApp = () => {
  const [todos, setTodos] = useState([]);

  // ADD: Create new array with spread operator
  const addTodo = (newText) => {
    const newTodo = { id: Date.now(), text: newText };
    setTodos([...todos, newTodo]);
  };

  // REMOVE: Filter out the specific ID
  const deleteTodo = (idToRemove) => {
    setTodos(todos.filter(todo => todo.id !== idToRemove));
  };

  // UPDATE: Map over and replace the specific item
  const updateText = (idToUpdate, newText) => {
    setTodos(todos.map(todo =>
      todo.id === idToUpdate ? { ...todo, text: newText } : todo
    ));
  };
};
```

STATE BEST PRACTICES

1. Group Related State

If state variables always change together (e.g., `firstName` and `lastName`), group them into a single object instead of using multiple `useState` calls. However, don't put unrelated data into the same object.

2. Avoid Redundant/Derived State

If a value can be calculated from existing state or props, calculate it during render; do not put it in state. For example, if you have `cartItems`, don't create a `cartTotal` state. Just calculate `const total = cartItems.reduce(...)` directly in the component body.

3. Keep State Local Where Possible

Don't lift state up higher in the component tree than it needs to go. If only a single form uses a piece of state, keep it inside that form component to avoid unnecessary re-renders of parent components.

✓ CHAPTER 6 SUMMARY

- **State** is mutable, component-internal data. **Props** are immutable, external data.
- `useState` schedules asynchronous updates. React batches these updates for performance.
- Always use **functional updates** (`prev ⇒ prev + 1`) if the new state depends on the old state.
- **Immutability is strictly required** for Objects and Arrays. Always create copies using the spread operator `...` or methods like `map/filter` before modifying.

INTERVIEW PREP

Q: What happens if I call `setState` inside a loop or after a timeout?

A: If you rely on the state variable directly (e.g., `count`) inside a `setTimeout` or interval, you might encounter a **"Stale Closure"**. The callback might "remember" the old value of `count` from when the timeout was created. To fix this, you must use the functional updater form: `setCount(prevCount ⇒ prevCount + 1)`, which ensures React provides the most recent state value at the time of execution.

07 EVENT HANDLING

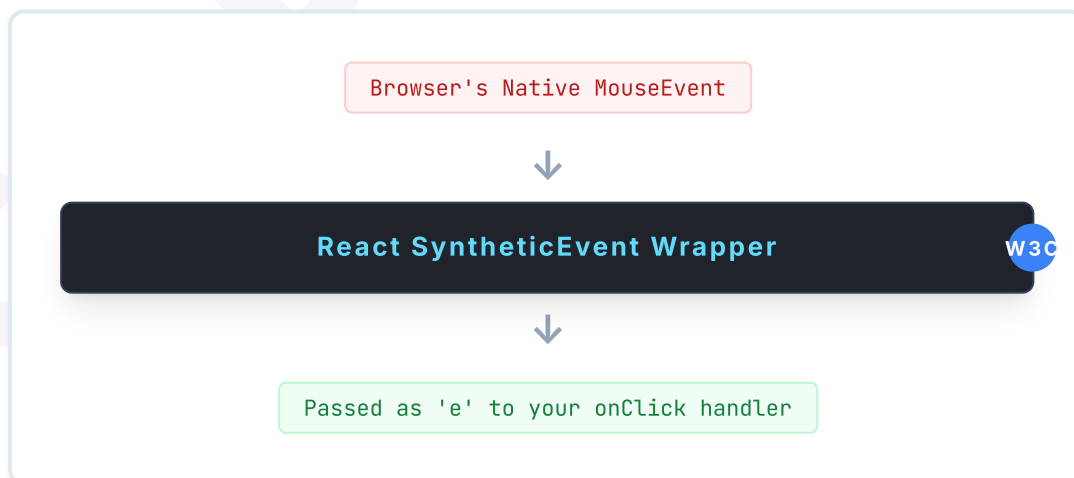
Event handling in React is very similar to handling events on DOM elements in plain JavaScript, but with a few critical syntactical and architectural differences. React normalizes events so they behave identically across all browsers.

React Events vs. HTML Events

Concept	HTML (Vanilla JS)	React (JSX)
Naming Convention	Lowercase (onClick)	camelCase (onClick)
Passing the Handler	Passed as a string ("myFunction()")	Passed as a function reference ({myFunction})
Preventing Default	Can return false;	Must explicitly call e.preventDefault()

The SyntheticEvent Object

When an event fires in React (like a click), the event handler doesn't receive the raw browser event. Instead, it receives a **SyntheticEvent**. This is a cross-browser wrapper created by React that conforms to the W3C spec. It guarantees that the event object has the same properties (like `e.stopPropagation()` and `e.preventDefault()`) regardless of whether the user is on Chrome, Safari, or Firefox.



Basic Event Example (Form Submission)

```
const FormComponent = () => {
  // 'e' is the SyntheticEvent
  const handleSubmit = (e) => {
    e.preventDefault(); // Stops page refresh
    console.log("Form submitted!");
  };

  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">Submit</button>
    </form>
  );
};
```

PASSING ARGUMENTS TO HANDLERS

The most common mistake beginners make in React is trying to pass an argument to an event handler exactly as they would in vanilla JavaScript, which causes the function to execute *immediately* when the component renders, rather than waiting for the user to click.

✗ THE INFINITE LOOP TRAP

```
// BAD: By adding (), you are CALLING the
// function right now during the render
// phase.
// If this function updates state, it will
// trigger another render, causing an
// infinite loop!
```

```
<button onClick={ deleteItem(id) }>
  Delete
</button>
```

✓ THE ARROW FUNCTION FIX

```
// GOOD: Wrap it in an anonymous arrow
// function.
// Now you are passing a FUNCTION REFERENCE
// that
// WILL call deleteItem(id) later, when
// clicked.
```

```
<button onClick={ () => deleteItem(id) }>
  Delete
</button>
```

Passing Both an Argument AND the Event

Sometimes you need to pass a specific piece of data (like an ID) to your handler, but you *also* need access to the SyntheticEvent object (e.g., to stop propagation or read a value). You can pass the event `e` through the arrow function.

```
const List = ({ items }) => {

  // Handler expects the ID first, then the Event
  const handleAction = (id, e) => {
    e.stopPropagation(); // e works perfectly!
    console.log(`Action performed on item ${id}`);
  };

  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>
          {item.name}
          {/* Pass 'e' from React down to handleAction */}
          <button onClick={ (e) => handleAction(item.id, e) }>
            Perform Action
          </button>
        </li>
      ))}
    </ul>
  );
};
```

INTERVIEW PREP

Q: In older React codebases using Class components, why do you see `this.handleClick.bind(this)` in the constructor?

A: In JavaScript, class methods are not bound by default. If you pass a method reference to an event handler without binding it, the `this` keyword inside that method will evaluate to `undefined` when the button is clicked, crashing the app. Modern React uses Functional Components and Arrow Functions, which lexically inherit `this` from their surrounding scope, eliminating the need for manual binding entirely.

08

CONDITIONAL RENDERING

In React, you can create distinct components that encapsulate behavior you need. Then, you can render only some of them, depending on the state of your application. Conditional rendering in React works exactly the same way conditions work in standard JavaScript.

The if Statement & Early Returns

The standard JavaScript `if` statement is perfect for **Early Returns**. If a condition is met (like data still loading, or an error occurring), you can return a completely different piece of JSX immediately, stopping the rest of the component from executing and rendering.

```
const UserProfile = ({ user, isLoading, hasError }) => {  
  
  // 1. Early Return for Loading State  
  if (isLoading) {  
    return <div className="spinner">Loading data...</div>;  
  }  
  
  // 2. Early Return for Error State  
  if (hasError || !user) {  
    return <div className="error">User not found!</div>;  
  }  
  
  // 3. Standard Return (Only reached if above conditions are false)  
  return (  
    <div className="profile-card">  
      <h1>{user.name}</h1>  
      <p>Email: {user.email}</p>  
    </div>  
  );  
};
```

⚠ Why can't I put an if statement inside JSX?

A common mistake is trying to write `<div> {if (isAdmin) { ... }} </div>`. This will immediately throw a syntax error.

Reason: JSX curly braces `{ }` can only contain JavaScript *Expressions* (code that resolves to a specific value). An `if` statement is a *Statement*, it performs an action but doesn't resolve to a value. To write inline conditionals inside JSX, we must use the Ternary Operator or Logical AND.

INLINE CONDITIONALS

1. The Ternary Operator (? :)

The ternary operator acts as an inline if-else statement. It is perfect for toggling between two different pieces of UI based on a boolean value.

```
condition ? renderThisIfTrue : renderThisIfFalse
```

```
const LoginButton = ({ isLoggedIn }) => {
  return (
    <div>
      {/* Renders Logout button if true, Login button if false */}
      {isLoggedIn ? (
        <button className="btn-red">Logout</button>
      ) : (
        <button className="btn-blue">Login</button>
      )}
    </div>
  );
};
```

2. Logical AND (&&)

Often, you don't need an else fallback. You just want to render an element if a condition is true, and render *nothing* if it is false. Using a ternary like `condition ? <Ui/> : null` works, but the logical `&&` (Short-circuit evaluation) is much cleaner.

```
condition && renderThisIfTrue
```

```
const Notifications = ({ count }) => {
  return (
    <div>
      <h2>Inbox</h2>
      {/* If count > 0 is true, the span renders. If false, React ignores it. */}
      {count > 0 && <span className="badge">{count} New</span>}
    </div>
  );
};
```



The Zero Rendering Bug

If you write `{messages.length && <Badge/>}` and the array is empty, `length` is `0`. Because `0` is a "falsy" value, JavaScript short-circuits and returns `0`. React will actually render the number "0" onto the screen!

The Fix: Always ensure the left side of the `&&` evaluates to a strict boolean. Use `messages.length > 0 && ...` or the double-bang `!!messages.length && ....`

HANDLING MULTIPLE CONDITIONS

When a component has more than two possible states (e.g., Pending, Approved, Rejected, Cancelled), nesting multiple ternary operators inside your JSX becomes an unreadable mess. React provides much cleaner patterns for this.

1. The switch Helper Function

Since you can't use `switch` directly inside JSX `{ }`, a common pattern is to abstract the switch logic into a helper function inside your component, and then call that function in the return statement.

```
const StatusBadge = ({ status }) => {

  // Helper function containing the logic
  const renderBadge = () => {
    switch (status) {
      case 'pending':
        return <span className="bg-yellow-500">Pending</span>;
      case 'approved':
        return <span className="bg-green-500">Approved</span>;
      case 'rejected':
        return <span className="bg-red-500">Rejected</span>;
      default:
        return <span className="bg-gray-500">Unknown</span>;
    }
  };

  return (
    <div>
      Application Status: {renderBadge()}
    </div>
  );
};
```

2. The Object Map (Dictionary) Pattern

A highly preferred, modern React pattern is to replace switch statements entirely using a standard JavaScript Object. You map the condition keys directly to the JSX elements you want to render. This results in incredibly clean, declarative code.

```
const StatusBadge = ({ status }) => {
  // 1. Define the dictionary map
  const badgeMap = {
    pending: <span className="bg-yellow-500">Pending</span>,
    approved: <span className="bg-green-500">Approved</span>,
    rejected: <span className="bg-red-500">Rejected</span>
  };

  // 2. Render using the key, with a logical OR (||) fallback for default
  return (
    <div>
      Application Status:
      {badgeMap[status] || <span className="bg-gray-500">Unknown</span>}
    </div>
  );
};
```

09

LISTS & KEYS

Most web applications involve displaying lists of data (e.g., a feed of tweets, a list of products, or a navigation menu). In traditional HTML/JS, you might use a `for` loop to manually create and append DOM elements. In React, because JSX can execute JavaScript expressions, we rely almost entirely on the built-in `Array map()` method.

Rendering Lists with `map()`

The `Array.prototype.map()` function creates a *new array* populated with the results of calling a provided function on every element in the calling array. In React, we use it to transform an array of data (strings, objects) into an array of JSX elements.

```
const frameworks = ['React', 'Angular', 'Vue', 'Svelte'];

const FrameworkList = () => {
  // We inject JS into JSX using { }
  return (
    <ul>
      {frameworks.map((framework) => (
        {/* Transforming each string into an <li> */}
        <li>{framework}</li>
      ))}
    </ul>
  );
};
```

❗ Why `map()` and not `forEach()` or `for-loops`?

JSX expressions `{ }` require you to provide something that evaluates to a value (an expression).

- A `for` loop is a *statement*. It doesn't return anything.
- `forEach()` iterates over an array but returns `undefined`.
- `map()` iterates over an array and **returns a new array**. React is smart enough to take an array of JSX elements and render them side-by-side automatically.



Warning: Each child in a list should have a unique "key" prop.

If you run the code above, it will work, but React will throw a glaring red warning in your browser console. This leads us to the most important rule of lists in React: **Keys**.

THE IMPORTANCE OF KEYS

When you render a dynamic list, items can be added, removed, or reordered. React needs a reliable way to identify exactly *which* item changed so it can update the Real DOM efficiently without re-rendering the entire list. **Keys** are special string attributes you must include when creating lists of elements.

```
const users = [
  { id: 101, name: 'Alice' },
  { id: 102, name: 'Bob' }
];

return (
  <ul>
    {users.map(user => (
      { /* The key MUST go on the outermost element returned by map */}
      <li key={user.id}>{user.name}</li>
    ))}
  </ul>
);
```

1

Unique among siblings

Keys only need to be unique among items in the same array. They do not need to be globally unique.

2

Stable (Not Random)

Never use `Math.random()` as a key. It changes every render, forcing React to destroy and recreate the DOM node.

3

Outermost Element

If you extract list items into a component, put the key on the `<Item/>` tag, not the `<li>` inside it.

🚨 ANTI-PATTERN: USING THE ARRAY INDEX AS A KEY

If you don't have unique IDs in your data, it's tempting to use the loop index provided by `map()`:

```
{items.map((item, index) => <li key={index}>{item.name}</li>)}
```

Why is this dangerous? If the list is completely static (never changes, reorders, or filters), using the index is okay. However, if users can add, delete, or sort items, the index of every subsequent item changes. React will get confused, causing state from one item to "bleed" into another, resulting in bizarre UI bugs (e.g., typing in the 1st input, deleting it, and the 2nd input suddenly has the 1st input's text). Always prefer database IDs or generate stable UUIDs (like `crypto.randomUUID()`) when the item is created.

DYNAMIC LISTS & BEST PRACTICES

Rendering Dynamic State

Lists in React are rarely static. They are usually tied directly to the component's state. When you combine `useState` with `map()`, any time you update the state array, React automatically re-renders the UI to match the new data.

TodoList.jsx

```
import { useState } from 'react';

const TodoList = () => {
  // 1. State holds the array of objects
  const [tasks, setTasks] = useState([
    { id: 1, text: 'Learn React' },
    { id: 2, text: 'Master Keys' }
  ]);

  // 2. Deletion function (filters array by ID)
  const handleDelete = (id) => {
    setTasks(tasks.filter(task => task.id !== id));
  };

  return (
    <ul>
      {tasks.map(task => (
        /* 3. Render dynamically. Key is crucial here! */
        <li key={task.id} className="flex justify-between">
          {task.text}
          <button onClick={() => handleDelete(task.id)}>
            Remove
          </button>
        </li>
      ))}
    </ul>
  );
};
```

Extracting List Items to Components

If the JSX inside your `map()` callback becomes complex (e.g., mapping over a list of users to display complex profile cards), it's highly recommended to extract the item UI into its own separate component.

```
// Parent Component
{users.map(user => (
  /* ✅ The Key MUST be on the custom component itself */
  <UserProfileCard key={user.id} user={user} />
))}

// Child Component (Does NOT need to receive 'key' as a prop)
const UserProfileCard = ({ user }) => {
  return <div className="card">... </div>;
};
```

10 FORMS

In standard HTML, form elements like `<input>`, `<textarea>`, and `<select>` maintain their own internal state and update it based on user input. In React, we generally want the React component's state to be the "single source of truth."

Controlled vs. Uncontrolled Components

STANDARD Controlled

React state completely controls the input. The input's `value` attribute is bound to a state variable, and an `onChange` handler updates that state on every keystroke.

```
// React knows the exact value at all times
<input
  value={name}
  onChange={(e) => setName(e.target.value)}
/>
```

ALTERNATIVE Uncontrolled

The DOM handles the form data directly (like vanilla JS). You use a React `useRef` hook to "reach in" and grab the value only when you need it (e.g., on submit).

```
// React does NOT track every keystroke
<input
  ref={nameRef}
  defaultValue="Bob"
/>
```

THE ANATOMY OF A CONTROLLED INPUT

If you set the `value` prop of an input to a state variable, but forget to add an `onChange` handler, the input becomes **read-only**. The user cannot type in it because React forces the input to constantly match the unchanging state variable.

```
const SimpleForm = () => {
  const [email, setEmail] = useState("");

  return (
    <input
      type="email"
      /* 1. Bind to State */
      value={email}
      /* 2. Update State on interaction */
      onChange={(e) => setEmail(e.target.value)}
    />
  );
};
```

STANDARD INPUT TYPES

React normalizes the API for `<input>`, `<textarea>`, and `<select>`. Unlike standard HTML where they all use slightly different syntax (e.g., inner text for `textarea`, `selected` attribute for `select`), in React, they all universally use the `value` attribute and `onChange` event.

The Unified State Object Pattern

Instead of creating 10 different `useState` hooks for a form with 10 fields, the industry standard is to use a single object. We utilize the input's `name` attribute to dynamically update the correct property in the state.

```
const ProfileForm = () => {
  // Single state object for all text-based fields
  const [formData, setFormData] = useState({
    username: "",
    bio: "",
    country: "us"
  });

  // Universal handler for inputs, textareas, and selects
  const handleChange = (e) => {
    const { name, value } = e.target; // Destructure for clarity
    setFormData(prev => ({
      ...prev, // Keep existing data
      [name]: value // Dynamically update the changed field
    }));
  };

  return (
    <form>
      {/* 1. Standard Input */}
      <input
        name="username" ← MUST match state key
        value={formData.username}
        onChange={handleChange}
      />

      {/* 2. Textarea (Note: No children, uses 'value' prop instead) */}
      <textarea
        name="bio"
        value={formData.bio}
        onChange={handleChange}
      />

      {/* 3. Select Dropdown (Value goes on the select, not the option) */}
      <select name="country" value={formData.country} onChange={handleChange}>
        <option value="us">United States</option>
        <option value="ca">Canada</option>
      </select>
    </form>
  );
};
```

TOGGLES: CHECKBOX & RADIO

Checkboxes (Boolean State)

Checkboxes require slightly different handling. Instead of reading `e.target.value`, a checkbox provides a boolean indicating if it is ticked or not via `e.target.checked`. Similarly, you bind its state to the `checked` attribute, not `value`.

```
const [agreed, setAgreed] = useState(false);

<input
  type="checkbox"
  checked={agreed}
  onChange={(e) => setAgreed(e.target.checked)}
/>
```

STATE VALUE

agreed: true/false

Radio Buttons (Grouped State)

Radio buttons are linked together by sharing the same name attribute. Only one state variable is needed for the entire group. You determine if a specific radio is active by checking if its `value` matches the current state variable.

```
const [paymentMethod, setPaymentMethod] = useState("card");

{/* Group Handler */}
const handleRadio = (e) => setPaymentMethod(e.target.value);

<label>
  <input
    type="radio" name="payment" value="card"
    {/* Evaluates to true if state === "card" */}
    checked={paymentMethod === "card"}
    onChange={handleRadio}
  /> Credit Card
</label>

<label>
  <input
    type="radio" name="payment" value="paypal"
    checked={paymentMethod === "paypal"}
    onChange={handleRadio}
  /> PayPal
</label>
```

SUBMISSION & BEST PRACTICES

Form Submission

To submit a form, you attach an `onSubmit` event handler directly to the `<form>` tag. It is absolutely crucial to call `e.preventDefault()` inside this handler. If you don't, the browser will attempt to send a standard HTTP POST request and refresh the entire page, completely erasing your React state and breaking the Single Page Application experience.

```
const handleSubmit = (e) => {
  // 1. Prevent standard browser page reload
  e.preventDefault();

  // 2. Validate data here
  if (formData.username.trim() === "") {
    setError("Username is required");
    return;
  }

  // 3. Make API Call (e.g., using fetch or Axios)
  console.log("Submitting to API:", formData);
};

return (
  /* Attach handler to the form, not the button */
  <form onSubmit={handleSubmit}>
    {error && <p className="text-red">{error}</p>}
    <input ... />
    /* Button type="submit" triggers the form's onSubmit */
    <button type="submit">Register</button>
  </form>
);
```

PRO TIP Advanced Form Libraries

As your application scales, creating controlled components manually for complex forms (with dozens of fields, complex validation rules, and nested data) becomes highly inefficient. Every keystroke triggers a re-render of the entire component.

Industry Standard: In professional environments (Level 4/5 of this roadmap), developers rely on third-party libraries like **React Hook Form** (which leverages uncontrolled components for maximum performance) paired with **Zod** or **Yup** for schema validation.

11 REACT HOOKS

Introduced in React 16.8, Hooks revolutionized how we write React. They are special functions that allow you to "hook into" React state and lifecycle features directly from functional components, entirely replacing the need for Class components.

⚠ THE TWO GOLDEN RULES OF HOOKS

- 1. Only Call Hooks at the Top Level:** Never call Hooks inside loops, conditions, or nested functions. React relies on the exact order in which Hooks are called to associate them with the correct local state.
- 2. Only Call Hooks from React Functions:** You can call them from React Functional Components, or from your own Custom Hooks. Never call them from regular vanilla JavaScript functions.

The useEffect() Hook

While `useState` allows a component to remember data, `useEffect` lets a component perform **Side Effects**. Side effects are actions that reach outside the React component, such as fetching data from an API, manually changing the DOM, or setting up subscriptions (like WebSockets or Timers).

```
useEffect(() => {
  // 1. The Effect Code (What you want to do)
  document.title = `You clicked ${count} times`;
}, [count]); // 2. The Dependency Array (When to do it)
```

The Dependency Array Matrix

Syntax	When does the effect run?
<code>useEffect(() => {...})</code>	No array: Runs after the initial render AND after <i>every single</i> re-render. Usually a bad idea (can cause infinite loops).
<code>useEffect(() => {...}, [])</code>	Empty array: Runs only once immediately after the component mounts. Perfect for initial API fetches.
<code>useEffect(() => {...}, [x, y])</code>	With dependencies: Runs on initial mount, and then re-runs <i>only if</i> the value of x or y has changed between renders.

EFFECT CLEANUP & REFS

The Cleanup Function

Some effects require cleanup to prevent memory leaks (e.g., removing an event listener, clearing a timeout, or aborting a network request). If your effect returns a function, React will run that cleanup function before the component unmounts (is destroyed), and right before it re-runs the effect on subsequent renders.

```
useEffect(() => {
  // 1. Setup the subscription / listener
  const timer = setInterval(() => {
    console.log("Tick...");
  }, 1000);

  // 2. Return a cleanup function
  return () => {
    // Runs when component unmounts
    clearInterval(timer);
  };
}, []);
```

The useRef() Hook

useRef is a "swiss army knife" hook that solves two very specific problems:

1. Accessing DOM Elements

Since React uses the Virtual DOM, we don't use `document.getElementById`. We attach a ref to directly interact with a real DOM node (e.g., to force focus).

```
const inputRef = useRef(null);

// Focus the input on mount
useEffect(() => {
  inputRef.current.focus();
}, []);



```

2. Storing Mutable Values

Unlike `useState`, changing a `useRef` value **does not trigger a re-render**. It's perfect for tracking things behind the scenes (like previous state or a stopwatch ID).

```
const renderCount = useRef(1);

useEffect(() => {
  // Mutate .current directly. No re-render!
  renderCount.current = renderCount.current +
  1;
});

{/* Displays the count silently */}
```

PERFORMANCE OPTIMIZATION HOOKS

Every time a component's state or props change, React re-renders it, running the entire function body from top to bottom. If you have expensive calculations or pass functions down to deeply nested children, you might need to memoize (cache) them to prevent performance bottlenecks.

The useMemo() Hook

useMemo is used to cache the **result** of a calculation. It will only recalculate the value if one of the variables in its dependency array has changed.

```
const ComplexChart = ({ data, darkTheme }) => {  
  // This calculation takes 500ms! We don't want it to run just because the user toggled the  
  theme.  
  const formattedData = useMemo(() => {  
    return slowFormattingLogic(data);  
  }, [data]); // ← Only re-runs if 'data' array changes  
};
```

The useCallback() Hook

In JavaScript, every time a functional component re-renders, it creates a brand new memory reference for any functions defined inside it. useCallback caches the **function definition itself**, preventing unnecessary re-renders of child components that accept that function as a prop.

```
const Parent = () => {  
  const [count, setCount] = useState(0);  
  
  // If we didn't use useCallback, <HeavyChild /> would re-render every time 'count' changes,  
  // because handleSubmit would be a "new" function every render.  
  const handleSubmit = useCallback((data) => {  
    apiCall(data);  
  }, []);  
  
  return <HeavyChild onSubmit={handleSubmit} />;  
};
```



Premature Optimization is the Root of All Evil

Do not wrap every function in useCallback or every variable in useMemo. Caching things takes memory and checking dependency arrays takes processing power. Most React renders are blazing fast. **Only use these hooks if you have proven performance issues** (using React Profiler) or if the value/function is being passed into a highly optimized child component (using React.memo).

ADVANCED STATE HOOKS

The useContext() Hook

Remember the **Prop Drilling** problem from Chapter 5? React Context is the solution. It allows you to create global variables that can be accessed by any component in the tree, bypassing props entirely. `useContext` is the hook that allows a child component to easily "consume" that global data.

```
import { useContext } from 'react';
import { ThemeContext } from './ThemeContext';

const DeeplyNestedCard = () => {
  // Magic! We grab the theme data without any props.
  const currentTheme = useContext(ThemeContext);

  return <div className={`card-${currentTheme}`}>Content</div>;
};
```

The useReducer() Hook

When you have complex state logic that involves multiple sub-values, or when the next state depends heavily on the previous state, `useState` becomes hard to manage. `useReducer` provides a centralized way to update state using a **Reducer function** and **Actions**. It operates on the exact same architecture as Redux.

```
// 1. Reducer Function (Outside Component)
const reducer = (state, action) => {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    case 'DECREMENT':
      return { count: state.count - 1 };
    default:
      return state;
  }
};
```

```
// 2. Inside Component
const Counter = () => {
  const [state, dispatch] = useReducer(
    reducer,
    { count: 0 } // Initial State
  );

  return (
    <button onClick={() => dispatch({type:
      'INCREMENT'})}>
      Add: {state.count}
    </button>
  )
};
```

SPECIALTY & CUSTOM HOOKS

The `useLayoutEffect()` Hook

The signature is identical to `useEffect`, but it fires synchronously **after all DOM mutations, but BEFORE the browser has painted the screen**. Use this exclusively for measuring DOM elements (like scroll position, or element width) and calculating visual updates to prevent the user from seeing a "flicker" of UI shifting.

Rule of thumb: 99% of the time, use `useEffect`.

Custom Hooks

Building your own Hooks lets you extract component logic into reusable functions. If you find yourself writing the exact same `useEffect` and `useState` logic across multiple components (e.g., fetching data, tracking window resize, handling forms), extract it into a Custom Hook.

Custom Hooks MUST always start with "use" (e.g., `useWindowSize`, `useFetch`).

```
// 1. Create the Custom Hook
const useWindowWidth = () => {
  const [width, setWidth] = useState(window.innerWidth);

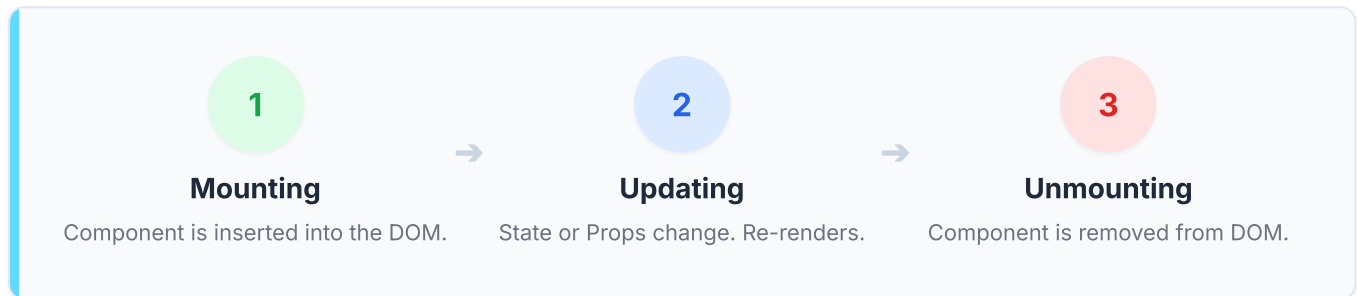
  useEffect(() => {
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return width; // It acts just like a regular function returning a value
};

// 2. Use it cleanly in any component!
const MobileMenu = () => {
  const width = useWindowWidth();
  return width < 768 ? <Hamburger /> : <FullNav />;
};
```

12 COMPONENT LIFECYCLE

Just like humans, React components go through a lifecycle. They are born (rendered into the DOM), they grow and change (update via state or props), and eventually, they die (are removed from the DOM). Understanding this lifecycle is crucial for initializing data, optimizing performance, and preventing memory leaks.



1. Mounting (Birth)

The mounting phase is when the component is created and inserted into the browser's DOM for the very first time. In legacy Class components, this was handled by `componentDidMount()`. In modern React, we use the `useEffect` hook with an **empty dependency array** `[]`.

```
const UserProfile = () => {
  const [userData, setUserData] = useState(null);

  // ● MOUNTING PHASE
  // The empty array [] guarantees this ONLY runs once.
  useEffect(() => {
    console.log("Component Mounted! Fetching data...");

    // Simulating an API Fetch on load
    fetchData().then(data => setUserData(data));
  }, []);

  return <div>{userData ? userData.name : 'Loading...'}</div>;
};
```

Primary uses for Mounting: Making initial API calls to fetch data, starting timers or intervals (like a clock), or initializing third-party libraries (like attaching a 3D Canvas context).

2. UPDATING (GROWTH)

A component updates whenever its **State** changes, its **Props** change, or its parent component re-renders. When an update occurs, React re-executes the component function to calculate the new UI.

In class components, this was handled by `componentDidUpdate()`. In modern React, we again use `useEffect`, but this time we provide specific variables in the **dependency array**.

WATCH SPECIFIC VARIABLE

```
// Runs on mount, AND whenever
// 'userId' changes.
useEffect(() => {
  console.log("User changed!");
  fetchUserDetails(userId);
}, [userId]);
```

WATCH EVERY RENDER (WARNING)

```
// Runs on mount, AND after
// ANY state or prop change.
// (No dependency array)
useEffect(() => {
  console.log("I run constantly!");
});
```

ADVANCED PATTERN: SKIPPING THE INITIAL MOUNT

A quirk of `useEffect` is that it *always* runs on the initial mount, even if you only want it to run on subsequent updates. To perfectly simulate `componentDidUpdate` (running **ONLY** on updates), we combine `useEffect` with `useRef`.

```
const isFirstRender = useRef(true);

useEffect(() => {
  if (isFirstRender.current) {
    isFirstRender.current = false; // Mark as mounted
    return; // Exit early on the first run
  }

  console.log("This ONLY runs on updates, not on birth!");
}, [searchTerm]);
```

3. UNMOUNTING (DEATH)

The unmounting phase occurs when a component is completely removed from the DOM (e.g., when a user navigates to a different page, or when a conditional rendering statement `isVisible && `` evaluates to false).

The Cleanup Function

To run code right before a component dies (formerly `componentWillUnmount()`), you **return a function** from inside your `useEffect`. This is essential to prevent memory leaks caused by lingering event listeners, websockets, or active timers.

```
const MouseTracker = () => {
  useEffect(() => {
    // 1. SETUP (Runs on mount)
    const logMouse = (e) => console.log(e.clientX, e.clientY);
    window.addEventListener('mousemove', logMouse);

    // 2. CLEANUP (Runs right before unmount)
    return () => {
      console.log("Component dying. Cleaning up listeners...");
      window.removeEventListener('mousemove', logMouse);
    };
  }, []);
};
```

Lifecycle: Class vs. Hooks Summary

Phase	Legacy Class Method	Modern Hook Equivalent
Mounting	<code>componentDidMount()</code>	<code>useEffect(..., [])</code>
Updating	<code>componentDidUpdate()</code>	<code>useEffect(..., [deps])</code>
Unmounting	<code>componentWillUnmount()</code>	<code>return () => { ... }</code> inside <code>useEffect</code>

13 STYLING REACT

React itself does not prescribe a specific way to style your components. Over the years, the community has developed numerous approaches, ranging from traditional stylesheets to advanced JavaScript-based solutions. Choosing the right styling strategy depends entirely on your project's scale and team preferences.

1. Inline CSS

As discussed in Chapter 3, inline styles in React are written as JavaScript objects rather than strings. Property names must be **camelCased** (e.g., `backgroundColor` instead of `background-color`).

```
// Note the double curly braces {{ }}
const Alert = () => (
  <div style={{ color: 'red', fontWeight: 'bold', padding: '10px' }}>
    Warning!
  </div>
);
```

2. Global CSS

The most traditional approach. You write a standard `.css` file and import it directly into your component. However, this has a massive drawback in React: **Styles are completely global**. If you define `.button` in `Header.css` and `.button` in `Footer.css`, they will collide and overwrite each other based on import order.

```
import './App.css'; // Imported globally
```

3. CSS Modules

CSS Modules solve the global collision problem. By naming your file `[name].module.css`, Vite/Webpack automatically scopes the class names locally to that specific component by generating unique hashes.

Button.module.css

```
.btnPrimary {
  background-color: blue;
  color: white;
}
```

Button.jsx

```
// Import it as an object named 'styles'
import styles from './Button.module.css';

const Btn = () => (
  <button className={styles.btnPrimary}>
    Click Me
  </button>
);
```

UTILITY FRAMEWORKS

4. Tailwind CSS (Industry Standard)

Tailwind CSS is currently the most popular way to style React applications. It is a "utility-first" framework. Instead of writing custom CSS, you apply pre-defined utility classes directly to your JSX elements. This document itself is built entirely using Tailwind!

Pros

- **No Context Switching:** You never have to leave your `.jsx`` file to write styles.
- **Zero CSS File Bloat:** You aren't creating hundreds of unused classes.
- **Built-in Design System:** Colors, spacing, and typography look great by default.
- **Easy Responsiveness:** Use prefixes like ``md:``, ``lg:``.

```
// Tailwind Example
<div className="bg-blue-500 text-white p-4
rounded-lg shadow-md hover:bg-blue-600
transition md:w-1/2">
  Tailwind Card
</div>
```

5. Bootstrap / React-Bootstrap

Bootstrap is a classic UI framework. While you *can* import raw Bootstrap CSS, the preferred way in React is to use a library like **React-Bootstrap**, which completely rebuilds Bootstrap's JavaScript components (like Modals or Accordions) as pure React components without relying on jQuery.

```
// 1. Install: npm install react-bootstrap bootstrap
// 2. Import CSS in main.jsx
import 'bootstrap/dist/css/bootstrap.min.css';

// 3. Use pre-built React components
import { Button, Card } from 'react-bootstrap';

const BootstrapExample = () => (
  <Card style={{ width: '18rem' }}>
    <Card.Body>
      <Card.Title>Card Title</Card.Title>
      <Card.Text>Some quick example text.</Card.Text>
      <Button variant="primary">Go somewhere</Button>
    </Card.Body>
  </Card>
);
```

ADVANCED STYLING TOOLS

6. Styled Components (CSS-in-JS)

Styled Components is a popular library that allows you to write actual CSS directly inside your JavaScript using ES6 tagged template literals. It creates a new React component with the styles attached to it, generating unique class names automatically to prevent collisions.

```
import styled from 'styled-components';

// Create a styled button component. Notice the backticks ``
const StyledButton = styled.button`
  background-color: ${props => props.primary ? 'blue' : 'white'};
  color: ${props => props.primary ? 'white' : 'blue'};
  padding: 10px 20px;
  border-radius: 5px;
  font-weight: bold;

  &:hover {
    opacity: 0.8;
  }
`;

const App = () => (
  <div>
    {/* Use it like a regular component. Styles are dynamic based on props! */}
    <StyledButton primary>Primary Action</StyledButton>
    <StyledButton>Secondary Action</StyledButton>
  </div>
);
```

7. Sass / SCSS

Sass is a CSS preprocessor that adds features like variables, nested rules, and mixins to standard CSS. Vite supports Sass out of the box; you just need to install it (`npm i sass`). You can write `.scss` files and import them just like regular CSS files, or use them as CSS Modules (`[name].module.scss`).

styles.scss

```
// Variables
$primary-color: #3498db;

.navbar {
  background: $primary-color;

  // Nesting
  a {
    color: white;
    &:hover {
      text-decoration: underline;
    }
  }
}
```

Sass is excellent for complex custom designs where you need heavy math, sophisticated color logic, or deep nesting, but it's increasingly being replaced by CSS-in-JS (Styled Components) or Utility frameworks (Tailwind) in modern React development due to component scoping.

14 ROUTING

Because React builds Single Page Applications (SPAs), it only ever loads one HTML file (index.html). To create the illusion of multiple pages (like /about or /contact), we use **Client-Side Routing**. The industry standard library for this is **React Router**.

How Client-Side Routing Works

React Router intercepts changes to the browser's URL. Instead of letting the browser send a request to the server for a new page, React Router stops the request, looks at the new URL, and dynamically swaps out the React components on the screen to match that URL. This results in instant, lightning-fast "page" transitions.

1. Setup and <BrowserRouter>

To use React Router, you must first install it: `npm install react-router`. The foundational component is the `BrowserRouter`. It uses the HTML5 History API to keep your UI in sync with the URL. It must wrap your entire application, usually inside `main.jsx`.

main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
// 1. Import BrowserRouter
import { BrowserRouter } from 'react-router';
import App from './App.jsx';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    {/* 2. Wrap the App component */}
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

DEFINING ROUTES & NAVIGATION

2. <Routes> and <Route>

Inside your `App.jsx`, you define your mapping between URLs and Components using `<Routes>` and `<Route>`. The `Routes` component acts as a switchboard, looking at the URL and rendering the *first* child `Route` that matches.

```
import { Routes, Route } from 'react-router';
import Home from './pages/Home';
import About from './pages/About';

const App = () => {
  return (
    <div>
      {/* Navigation goes outside Routes to appear on every page */}
      <Navbar />

      <Routes>
        {/* The 'element' prop takes the JSX you want to render */}
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        {/* Catch-all route for 404 Not Found pages */}
        <Route path="*" element={<NotFound />} />
      </Routes>
    </div>
  );
};
```

3. <Link> vs <NavLink>

Never use standard `<a href="...">` tags in React! They tell the browser to request a completely new page from the server, breaking the SPA and wiping your state. Instead, use React Router's `<Link>` component.

Link Component

Used for standard, silent navigation.

```
<Link to="/about">
  About Us
</Link>
```

NavLink Component

Special type of `Link` used in Navigation bars. It automatically knows if it is "active" based on the URL.

```
<NavLink
  to="/about"
  className={({ isActive }) =>
    isActive ? "text-blue-500" : "text-gray"
  }
>About Us</NavLink>
```

DYNAMIC ROUTES & URL PARAMETERS

You can't hardcode a separate `<Route>` for every single product in an e-commerce store (e.g., `/product/1`, `/product/2`). Instead, you define a **Dynamic Route** using a URL Parameter.

1. Defining the Dynamic Segment

To create a dynamic segment in a path, prefix the segment with a colon (:).

```
// App.jsx
<Routes>
  { /* This route matches /product/apple, /product/123, etc. */ }
  <Route path="/product/:productId" element={<ProductDetails />} />
</Routes>
```

2. Extracting the Parameter with useParams()

Inside the component that renders (e.g., `ProductDetails`), you use the `useParams` hook to grab the value from the URL so you know which product data to fetch.

```
// ProductDetails.jsx
import { useParams } from 'react-router';
import { useEffect, useState } from 'react';

const ProductDetails = () => {
  // The key MUST match the dynamic segment name in the Route path
  const { productId } = useParams();
  const [product, setProduct] = useState(null);

  useEffect(() => {
    // Fetch specific product data when the component loads
    fetch(`https://api.store.com/products/${productId}`)
      .then(res => res.json())
      .then(data => setProduct(data));
  }, [productId]); // Add to dependency array!

  return <div>Displaying product ID: {productId}</div>;
};
```

Important Note: URL parameters extracted via `useParams()` are always returned as **Strings**. If you expect an ID to be a number (for comparison like `productId === 5`), you must convert it manually using `parseInt(productId)` or `Number(productId)`.

NESTED ROUTES & LAYOUTS

Often, URLs share parts of a UI. For example, navigating from `/dashboard/profile` to `/dashboard/settings` should keep the dashboard sidebar visible, only changing the main content area. This is achieved using **Nested Routes** and the `<Outlet>` component.

1. Defining Nested Routes

In your route configuration, you put child `<Route>` tags *inside* the parent `<Route>` tag.

```
<Routes>
  { /* Parent Route: Defines the Layout */ }
  <Route path="/dashboard" element={ <DashboardLayout /> }>

    { /* Child Routes (Notice relative paths, no leading slash) */ }
    { /* index route renders at exactly /dashboard */ }
    <Route index element={ <DashboardHome /> } />
    { /* Matches /dashboard/settings */ }
    <Route path="settings" element={ <Settings /> } />

  </Route>
</Routes>
```

2. The `<Outlet>` Component

The parent component (`DashboardLayout`) determines *where* the child components should render on the screen using the `<Outlet />` component as a placeholder.

DashboardLayout.jsx

```
import { Outlet } from 'react-router';

const DashboardLayout = () => {
  return (
    <div className="flex">
      { /* Fixed Sidebar */ }
      <Sidebar />

      <main className="flex-1 p-4">
        { /* Child route injects here */ }
        <Outlet />
      </main>
    </div>
  );
};
```

Sidebar (Static)

`<Outlet />`

Swaps between Home, Settings, etc. based on URL.

PROTECTED ROUTES & NAVIGATION

Protected Routes

In real applications, some routes (like /dashboard) should only be accessible if the user is authenticated. We achieve this by creating a wrapper component that checks the authentication state. If logged in, it renders the child content via `<Outlet>`. If not, it redirects the user to the login page using the `<Navigate>` component.

```
// ProtectedRoute.jsx
import { Navigate, Outlet } from 'react-router';
import { useAuth } from '../hooks/useAuth';

const ProtectedRoute = () => {
  const { isAuthenticated } = useAuth(); // Custom hook checking context/storage

  // If not logged in, force navigation to login page.
  // 'replace' prevents the user from going back to the protected route via browser history.
  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  // If logged in, render the requested nested route
  return <Outlet />;
};
```

Usage: `<Route element={}<ProtectedRoute/>> <Route path="/dashboard" ... /> </Route>`

Programmatic Navigation

While `<Link>` is declarative (user clicks it), sometimes you need to navigate the user programmatically after an action completes (like a successful form submission). For this, we use the `useNavigate` hook.

```
import { useNavigate } from 'react-router';

const LoginForm = () => {
  const navigate = useNavigate();

  const handleLogin = async (e) => {
    e.preventDefault();
    await performLoginAPI();

    // After successful login, redirect to dashboard
    navigate("/dashboard");
  };
};
```

15 API INTEGRATION

React is strictly a front-end library; it only handles the UI. To get dynamic data (like a list of users, products, or tweets) or to save user input to a database, React must communicate with a Backend Server via an Application Programming Interface (**API**), typically using JSON over HTTP.

1. The Modern Standard: `async/await`

Network requests take time. JavaScript is single-threaded, meaning it cannot pause execution to wait for a server response without freezing the entire browser. Historically, this was solved with Callbacks, then Promises (`.then().catch()`). Today, the modern standard is the cleaner, more readable `async/await` syntax.

Legacy: Promises

```
const getUser = () => {
  fetch('/api/user')
    .then(res => res.json())
    .then(data => console.log(data))
    .catch(err => console.error(err));
};
```

Modern: Async/Await

```
const getUser = async () => {
  try {
    const res = await fetch('/api/user');
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error(err);
  }
};
```

2. Fetch API vs. Axios

There are two primary ways to make HTTP requests in React. **Fetch** is built directly into all modern browsers (no installation required). **Axios** is a popular third-party library that sits on top of the native XMLHttpRequest and provides a cleaner, more feature-rich API.

Fetch API (Native)	Axios (Library)
Built-in; no installation required.	Requires <code>npm install axios</code> .
Does NOT throw errors on 404 or 500 status codes. You must check <code>res.ok</code> manually.	Automatically throws an error for any status code outside the 2xx range.
Requires a manual 2nd step to parse JSON: <code>await res.json()</code> .	Automatically parses JSON data into <code>res.data</code> .
Setting headers and stringifying body data is manual and verbose.	Shorthand methods, global interceptors, and automatic object serialization.

GET REQUESTS & COMPONENT STATE

A GET request is used to retrieve data from a server. Because data fetching is a "Side Effect" (an operation reaching outside the component), it **must** occur inside a `useEffect` hook, typically with an empty dependency array `[]` so it runs exactly once when the component mounts.

The "Holy Trinity" of Fetch State

A robust React component always manages three distinct pieces of state when fetching data to ensure a smooth user experience.

```
import { useState, useEffect } from 'react';

const UserList = () => {
  // 1. The Holy Trinity of State
  const [data, setData] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  // 2. Trigger fetch on Mount
  useEffect(() => {
    // Define async function inside useEffect
    const fetchUsers = async () => {
      try {
        const response = await fetch('https://jsonplaceholder.typicode.com/users');
        if (!response.ok) throw new Error("Failed to fetch data");

        const jsonData = await response.json();
        setData(jsonData);
      } catch (err) {
        setError(err.message);
      } finally {
        setIsLoading(false); // Always stop loading, success or fail
      }
    };
    fetchUsers();
  }, []); // ← Empty array means "Run once on mount"

  // 3. Conditional Rendering based on state
  if (isLoading) return <div>Loading...</div>;
  if (error) return <div className="text-red-500">Error: {error}</div>;

  return (
    <ul>{data.map(u => <li key={u.id}>{u.name}</li>)}</ul>
  );
};
```

MUTATING DATA (POST, PUT, DELETE)

When you want to send data *to* the server (creating, updating, or deleting records), you don't use `useEffect`. Instead, these mutations happen in response to user events, like clicking a "Submit" button or a "Delete" icon inside an event handler.

POST Requests (Creating Data)

To send data via the native Fetch API, you must explicitly pass a configuration object specifying the method, the headers (to tell the server you are sending JSON), and you must `JSON.stringify()` the JavaScript object into a string for the body.

```
const CreatePost = () => {
  const [title, setTitle] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();
    const newPost = { title: title, userId: 1 };

    try {
      const res = await fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        // Must convert JS object to JSON string
        body: JSON.stringify(newPost)
      });

      const savedData = await res.json();
      console.log("Server saved:", savedData);
    } catch (err) {
      console.error(err);
    }
  };

  // ... return <form onSubmit={handleSubmit}> ...
};
```

PUT / PATCH (Updating)

Identical to POST, but you use method: 'PUT' and append the specific resource ID to the URL endpoint.

```
await fetch(`/api/posts/${id}`, {
  method: 'PUT',
  body: JSON.stringify(updates)
});
```

DELETE (Removing)

Typically does not require a body or headers. Just specify the method and the ID in the URL.

```
await fetch(`/api/posts/${id}`, {
  method: 'DELETE'
});
```

AXIOS IN REACT

While the native Fetch API works fine, as applications grow, the manual boilerplate (checking `res.ok`, parsing JSON, stringifying bodies) becomes tedious. The industry strongly favors **Axios** for professional projects.

Why Axios is Cleaner

Axios provides convenience methods like `axios.get()` and `axios.post()`. It automatically serializes JavaScript objects to JSON, automatically parses JSON responses into a `data` property, and triggers the catch block automatically on HTTP errors (like 404 or 500).

```
// npm install axios
import axios from 'axios';

const AxiosDemo = () => {
  // GET Request Example
  const fetchUsers = async () => {
    try {
      // No need for .json(), data is ready immediately
      const res = await axios.get('/api/users');
      console.log(res.data);
    } catch (err) {
      // Auto-catches 404s and 500s!
      console.error(err.response.data.message);
    }
  };

  // POST Request Example
  const createPost = async (postObj) => {
    try {
      // No stringify! No manual headers! Just pass the object.
      const res = await axios.post('/api/posts', postObj);
    } catch (err) {
      console.error(err);
    }
  };
};
```

Axios Instances (Base URL)

Instead of typing `https://api.myapp.com/v1/ ...` in every single component, you can create a centralized Axios instance. This allows you to set a Base URL and attach things like Authorization Tokens globally.

```
// api.js (Centralized setup file)
import axios from 'axios';

const api = axios.create({
  baseURL: 'https://api.backend.com/v1',
  headers: { 'Authorization': 'Bearer my_token_123' }
});

export default api;

// In your component:
// Automatically prepends baseURL and adds the Token!
const res = await api.get('/users');
```

16 CONTEXT API

React's strict top-down data flow (passing props) works well for small apps. But as seen in Chapter 5, **Prop Drilling** occurs when you pass data through many intermediate components that don't need it. The **Context API** solves this by providing a way to share data globally across the entire component tree, completely bypassing props.

PROP DRILLING (WITHOUT CONTEXT)

App → Nav → Menu →
UserBadge

Data travels through every layer.

CONTEXT API

App → (Portal) →
UserBadge

Data jumps directly to where it's needed.

1. createContext()

The first step is to create the context itself. You usually do this in a separate file so it can be imported anywhere. `createContext()` returns a Context object.

```
// AuthContext.js
import { createContext } from 'react';

// Create context with an optional default value (usually null or empty object)
export const AuthContext = createContext(null);
```

2. The <Provider> Component

Every Context object comes with a built-in Provider component. You must wrap the part of your application that needs the global data inside this Provider. The Provider accepts a `value` prop; whatever you pass here becomes the global data.

```
// App.jsx
import { useState } from 'react';
import { AuthContext } from './AuthContext';
import MainLayout from './MainLayout';

const App = () => {
  // This state will be shared globally
  const [user, setUser] = useState({ name: 'Alice', loggedIn: true });

  return (
    /* Provide the data down the tree */
    <AuthContext.Provider value={{ user, setUser }}>
      <MainLayout />
    </AuthContext.Provider>
  );
};
```

CONSUMING CONTEXT

3. Legacy <Consumer> vs. Modern useContext()

Before React Hooks existed, to read context data inside a child component, you had to use the `<Context.Consumer>` component, which required an awkward "render props" pattern (passing a function as a child). Today, this is entirely replaced by the much cleaner `useContext()` hook.

❌ LEGACY CONSUMER

```
import { AuthContext } from './AuthContext';

const Header = () => (
  <AuthContext.Consumer>
    {value => (
      <div>Welcome, {value.user.name}</div>
    )}
  </AuthContext.Consumer>
);
```

✅ MODERN USECONTEXT

```
import { useContext } from 'react';
import { AuthContext } from './AuthContext';

const Header = () => {
  // Destructure directly!
  const { user } = useContext(AuthContext);

  return <div>Welcome, {user.name}</div>;
};
```

Updating Global State

Because we passed the `setUser` function down in the Provider's `value` prop, any component deep in the tree can call it to update the global state. When the state in the Provider updates, **all components consuming that context will automatically re-render** with the new data.

```
// Deeply nested LogoutButton.jsx
import { useContext } from 'react';
import { AuthContext } from '../AuthContext';

const LogoutButton = () => {
  // Grab the setter from context
  const { setUser } = useContext(AuthContext);

  const handleLogout = () => {
    // This updates the state back in App.jsx!
    setUser(null);
  };

  return <button onClick={handleLogout}>Log Out</button>;
};
```

BEST PRACTICE: THE CONTEXT PROVIDER PATTERN

Instead of cluttering `App.jsx` with multiple state variables and raw `<Context.Provider>` tags, the industry standard is to encapsulate the State logic, the Provider, and a custom Hook into a single file. This is commonly seen with **Theme Management** (Light/Dark mode).

ThemeContext.jsx (The complete package)

```
import { createContext, useState, useContext } from 'react';

// 1. Create Context (Not exported, kept private to this file)
const ThemeContext = createContext();

// 2. Create the Provider Component
export const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme(prev => prev === 'light' ? 'dark' : 'light');
  };

  return (
    /* Expose the state and the action function */
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};

// 3. Create a Custom Hook for easy consumption
export const useTheme = () => useContext(ThemeContext);
```

Usage: App.jsx

```
import { ThemeProvider } from './ThemeContext';

const App = () => (
  <ThemeProvider>
    <Navbar />
    <MainContent />
  </ThemeProvider>
);
```

Usage: Navbar.jsx

```
import { useTheme } from './ThemeContext';

const Navbar = () => {
  // Clean and simple!
  const { theme, toggleTheme } = useTheme();

  return (
    <button onClick={toggleTheme}>
      Current: {theme}
    </button>
  );
};
```

17

ADVANCED STATE MGMT

While the **Context API** is fantastic for simple global data (like themes or auth status), it has a major performance flaw: *Whenever the context value changes, EVERY component consuming that context re-renders.* For high-frequency, complex state (like a live shopping cart or chat app), we need dedicated state management libraries that allow fine-grained subscriptions.

Redux Toolkit (RTK)

Redux is the most famous state management library for React. Historically, it was notorious for requiring massive amounts of complex boilerplate code. Today, the official standard is **Redux Toolkit (RTK)**, which abstracts away the complexity and makes writing Redux fast and enjoyable.

```
npm install @reduxjs/toolkit react-redux
```

1. The Redux Store

The **Store** is the central vault that holds all the global state for your entire application. You only ever have *one* store per application. You create it using RTK's `configureStore`, and then provide it to your app using the `<Provider>` component from `react-redux`.

store.js

```
import { configureStore } from
  '@reduxjs/toolkit';
// We will create this slice in the next step
import cartReducer from './cartSlice';

export const store = configureStore({
  reducer: {
    // Registers the slice to the global store
    cart: cartReducer
  }
});
```

main.jsx

```
import { Provider } from 'react-redux';
import { store } from './store';
import App from './App';

// Wrap the App with the Redux Provider
root.render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

2. REDUX SLICES

A **Slice** is a collection of Redux reducer logic and actions for a single feature in your app (e.g., a "cart" slice, a "user" slice). RTK provides the `createSlice()` function, which automatically generates action creators and action types that correspond to the reducers you write.

```
// cartSlice.js
import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart', // Internal name used by Redux
  initialState: { items: [], totalItems: 0 },

  reducers: {
    // Reducer 1: Add Item
    addItem: (state, action) => {
      // 🚨 LOOK HERE! We are pushing directly to the array!
      state.items.push(action.payload);
      state.totalItems += 1;
    },
    // Reducer 2: Clear Cart
    clearCart: (state) => {
      state.items = [];
      state.totalItems = 0;
    }
  }
});

// Export the ACTIONS for components to use
export const { addItem, clearCart } = cartSlice.actions;
// Export the REDUCER for the store.js to use
export default cartSlice.reducer;
```

THE MAGIC OF IMMER.JS

Wait... in Chapter 6, we learned that React State **MUST** be strictly immutable. You can't use `push()` or modify `state.totalItems += 1` directly!

The Exception: Redux Toolkit uses a library called **Immer** under the hood. Immer wraps your state in a special "Draft" proxy. It *allows* you to write code that looks like it's mutating state (which is much easier to read and write), and then Immer silently translates those mutations into safe, immutable copies before applying them to the store.

Note: This magic ONLY works inside RTK's `createSlice`, not in standard React `useState`.

CONSUMING REDUX STATE

Now that our store and slices are configured, components need a way to interact with them. React-Redux provides two essential hooks: `useSelector` (to read data) and `useDispatch` (to update data).

1. READ (USESELECTOR)

Extracts specific data from the Redux store state. The component will automatically re-render only when the selected data changes.

```
const total = useSelector(state =>
state.cart.totalItems);
```

2. WRITE (USEDISPATCH)

Returns the dispatch function. You use this to send "Actions" (the ones exported from your slice) to the store.

```
const dispatch = useDispatch();
dispatch(addItem({ id: 1 }));
```

Putting it all together

```
// ProductCard.jsx
import { useSelector, useDispatch } from 'react-redux';
import { addItem, clearCart } from './cartSlice';

const ProductCard = ({ product }) => {
  // 1. Setup Dispatch
  const dispatch = useDispatch();

  // 2. Read state
  const totalItems = useSelector((state) => state.cart.totalItems);

  // 3. Dispatch an action with a payload
  const handleAddToCart = () => {
    dispatch(addItem(product));
  };

  return (
    <div className="card">
      <h3>{product.name}</h3>
      <p>Cart contains: {totalItems} items</p>

      <button onClick={handleAddToCart}>Add to Cart</button>
      <button onClick={() => dispatch(clearCart())}>Empty Cart</button>
    </div>
  );
};
```

MIDDLEWARE & DEVTOOLS

Middleware & Async Logic (Thunks)

Redux Reducers must be **100% pure functions**. This means they cannot contain side effects like `setTimeout`, DOM manipulations, or API fetch calls. To handle asynchronous logic, Redux uses **Middleware**—code that intercepts actions before they reach the reducer. RTK includes a middleware called **Redux Thunk** out of the box.

```
// usersSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

// 1. Define the Async Thunk
export const fetchUsers = createAsyncThunk('users/fetch', async () => {
  const response = await fetch('/api/users');
  return response.json(); // Returns the payload
});

// 2. Handle the lifecycle states in extraReducers
const usersSlice = createSlice({
  name: 'users',
  initialState: { data: [], status: 'idle' },
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => { state.status = 'loading'; })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.data = action.payload;
      })
      .addCase(fetchUsers.rejected, (state) => { state.status = 'failed'; });
  }
});
```

Redux DevTools

Time-Travel Debugging



One of Redux's greatest superpowers is the **Redux DevTools Extension** for Chrome/Firefox. Because state is immutable and actions are explicitly dispatched, the DevTools record a complete history of every state change. You can literally scrub backward and forward in time to see exactly how your UI responds to past states, without reloading the browser.

RTK configures this connection automatically!

ZUSTAND (THE MODERN ALTERNATIVE)

While Redux Toolkit is the enterprise standard, many developers still find it too verbose for smaller to medium projects. Enter **Zustand** (German for "state"). It is a small, fast, and scalable bare-bones state-management solution using simplified Flux principles.

Redux Toolkit Constraints

- Requires a <Provider> wrapper.
- Strict separation of Reducers and Actions.
- Requires complex Thunks for async logic.

Zustand Superpowers

- **No Provider wrapper required!**
- Actions and State live side-by-side.
- Async actions are just standard `async` functions.

Creating and Using a Zustand Store

```
// 1. store.js
import { create } from 'zustand';

// The set function allows you to merge state
export const useStore = create((set) => ({
  // State
  bears: 0,

  // Actions
  increasePopulation: () => set((state) => ({ bears: state.bears + 1 })),
  removeAllBears: () => set({ bears: 0 })
}));

// 2. BearCounter.jsx (Any component, anywhere)
import { useStore } from './store';

const BearCounter = () => {
  // Select specifically what you need for optimal re-renders
  const bears = useStore(state => state.bears);
  const increase = useStore(state => state.increasePopulation);

  return (
    <div>
      <h1>{bears} bears around here...</h1>
      <button onClick={increase}>Add Bear</button>
    </div>
  );
};
```

18 PERFORMANCE OPTIMIZATION

React is naturally fast due to its Virtual DOM. However, as applications scale in complexity, unnecessary re-renders or loading massive bundles of code upfront can severely impact performance. Optimization in React is about minimizing "wasted" renders and managing bundle sizes.

1. Component Memoization: `React.memo()`

By default, if a parent component re-renders, *all* of its child components will also re-render, even if the props passed to the children haven't changed. `React.memo()` is a Higher Order Component (HOC) that "memoizes" (caches) a functional component. It tells React: "Only re-render this component if its props have actually changed."

```
import { useState, memo } from 'react';

// 1. Wrap the child component in memo()
const ExpensiveChild = memo(({ text }) => {
  console.log("ExpensiveChild rendered!");
  return <div>{text}</div>;
});

const Parent = () => {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("Hello");

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Render Parent</button>

      {/* Because ExpensiveChild is memoized, clicking the button above */}
      {/* WILL NOT cause it to re-render, because the 'text' prop hasn't changed. */}
      <ExpensiveChild text={text} />
    </div>
  );
};
```

! The Prop Reference Trap

`React.memo` does a *shallow comparison* of props. If you pass an object, an array, or a function as a prop, `React` creates a new memory reference for them on every parent render. `React.memo` will see the new reference, assume the prop has changed, and re-render anyway, completely defeating the purpose of memoization! This is where `useMemo` and `useCallback` come in.

CACHING VALUES & FUNCTIONS

2. useMemo()

`useMemo` caches the *result* of a calculation between renders. It is used for two primary reasons: 1. To avoid running expensive, slow calculations on every render. 2. To cache the memory reference of an Object or Array so that a child wrapped in `React.memo` doesn't re-render unnecessarily.

```
const Parent = ({ data }) => {
  // 1. Expensive Calculation Caching
  // This sort function only runs when 'data' changes, not on every render.
  const sortedData = useMemo(() => {
    return [...data].sort((a, b) => b.val - a.val);
  }, [data]);

  // 2. Reference Caching for Props
  // 'config' keeps the same memory address across renders
  const config = useMemo(() => ({ theme: 'dark' }), []);

  return <MemoizedChild settings={config} items={sortedData} />;
};
```

3. useCallback()

Similar to `useMemo`, but specifically designed to cache the *function definition itself*. If you pass a callback function (like an `onClick` handler) down to a memoized child component, you *must* wrap it in `useCallback` to preserve its reference, otherwise the child will re-render every time.

```
const Parent = () => {
  const [count, setCount] = useState(0);

  // The 'handleAction' function definition is cached.
  // It will only be re-created if the dependencies change.
  const handleAction = useCallback((id) => {
    console.log("Action performed on:", id);
  }, []);

  return (
    <div>
      {/* This child won't re-render when 'count' changes because */}
      {/* 'handleAction' reference remains stable. */}
      <MemoizedButton onAction={handleAction} />
    </div>
  );
};
```

LAZY LOADING & CODE SPLITTING

By default, bundlers like Vite or Webpack package your entire React application into a single, massive JavaScript file. If your app has 50 pages, a user visiting the homepage has to download the code for the other 49 pages before they can see anything. **Code Splitting** divides the bundle, and **Lazy Loading** fetches those pieces only when needed.

4. React.lazy() and <Suspense>

You can dynamically import components using `React.lazy()`. When React encounters a lazy-loaded component, it suspends rendering until the code is downloaded. You must wrap lazy components in a `<Suspense>` boundary to display a fallback UI (like a spinner) while the code is fetching.

```
import { lazy, Suspense } from 'react';
import { Routes, Route } from 'react-router';

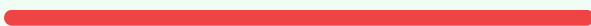
// 1. Standard Import (Bundled immediately)
import Home from './pages/Home';

// 2. Lazy Imports (Downloaded only when visited)
const Dashboard = lazy(() => import('./pages/Dashboard'));
const Settings = lazy(() => import('./pages/Settings'));

const App = () => {
  return (
    /* 3. Suspense Boundary provides the loading UI */
    <Suspense fallback=<div>Loading page code...</div>>
      <Routes>
        <Route path="/" element=<Home /> />
        /* The heavy dashboard is now isolated in its own JS chunk */
        <Route path="/dashboard" element=<Dashboard /> />
      </Routes>
    </Suspense>
  );
};
```

WITHOUT LAZY LOADING

Initial Load: User downloads bundle.js (2.5 MB)



Slow Time-to-Interactive (TTI)

WITH LAZY LOADING

Initial Load: main.js (300 KB)



Fast TTI. Chunks load on demand.

VIRTUALIZATION & BEST PRACTICES

5. List Virtualization

If you try to map over an array of 10,000 items and render them as DOM nodes, the browser will freeze and crash. **Virtualization** (or "Windowing") solves this by only rendering the 15-20 items that are currently visible within the user's viewport. As the user scrolls, the nodes are recycled and updated with new data. The industry standard library for this is `react-window` or `@tanstack/react-virtual`.

```
// npm install react-window
import { FixedSizeList } from 'react-window';

const MassiveList = ({ items }) => {
  // The Row component rendered for each visible item
  const Row = ({ index, style }) => (
    /* 'style' handles the absolute positioning required for virtualization */
    <div style={style}>Item {items[index].name}</div>
  );

  return (
    <FixedSizeList
      height={400} {/* Height of the scrollable container */}
      itemCount={items.length} {/* Total items (e.g., 10,000) */}
      itemSize={35} {/* Height of each individual row */}
      width={"100%"}
    >
      {Row}
    </FixedSizeList>
  );
};
```

⚡ PERFORMANCE GOLDEN RULES

- **Profile Before Optimizing:** Don't blindly add `useMemo` everywhere. Use the **React DevTools Profiler** tab to record interactions and identify components that take a long time to render.
- **State Colocation:** Move state as close as possible to where it's needed. If a modal's `isOpen` state is in the root `App`, toggling it re-renders the entire application. Move it down.
- **Avoid Anonymous Objects in Props:** Passing `style={{ color: 'red' }}` creates a new object reference on every render, triggering unnecessary re-renders in memoized children. Extract static objects/functions outside the component.

19 ERROR HANDLING

In a production application, errors are inevitable. A network request might fail, a server might return unexpected data, or a user might enter invalid input. Robust error handling ensures your application doesn't completely crash (the dreaded "white screen of death") and provides clear feedback to the user.

1. Error Boundaries

If an error is thrown inside a React component during rendering (e.g., trying to map over `undefined`), it will corrupt React's internal state and unmount the entire component tree. **Error Boundaries** are React components that catch JavaScript errors anywhere in their child component tree, log those errors, and display a fallback UI instead of crashing the whole app.

Crucial Note: As of React 18, Error Boundaries *must* be written as Legacy **Class Components**. There is currently no functional component Hook (like `useErrorBoundary`) that can catch rendering errors. However, the community standard is to use the `react-error-boundary` package, which provides a highly functional, easy-to-use wrapper.

```
// npm install react-error-boundary
import { ErrorBoundary } from 'react-error-boundary';

// 1. Create a Fallback Component
const ErrorFallback = ({ error, resetErrorBoundary }) => {
  return (
    <div role="alert" className="p-4 bg-red-100 text-red-900">
      <p>Something went wrong:</p>
      <pre>{error.message}</pre>
      {/* This allows the user to try again without a full page reload */}
      <button onClick={resetErrorBoundary}>Try again</button>
    </div>
  );
};

// 2. Wrap risky components
const App = () => (
  <ErrorBoundary fallbackComponent={ErrorFallback}>
    <BuggyDashboardWidget />
  </ErrorBoundary>
);
```

Note: Error Boundaries do NOT catch errors inside event handlers (like `onClick`), asynchronous code (like `setTimeout` or `fetch`), or errors thrown in the boundary itself. They only catch errors thrown during the render phase.

ASYNC & API ERRORS

2. Try-Catch for Async Operations

Because Error Boundaries don't catch errors in event handlers or async code, you must use standard JavaScript try ... catch blocks around your API calls and complex logic.

```
const submitForm = async (data) => {
  setIsSubmitting(true);
  setServerError(null); // Clear previous errors

  try {
    // Potentially risky operation
    await api.post('/checkout', data);
    navigate('/success');
  } catch (error) {
    // Safely handle the failure
    setServerError(error.message || "An unexpected error occurred.");
  } finally {
    // Always runs, resetting the loading state
    setIsSubmitting(false);
  }
};
```

3. Comprehensive API Error Handling

When working with an API (especially using Axios), errors aren't just strings; they are complex objects containing status codes and backend-provided messages. A robust application interprets these codes to give the user actionable feedback.

Advanced Axios Catch Block

```
catch (error) {
  if (error.response) {
    // 1. Server responded with a status code outside 2xx (e.g., 400, 401, 500)
    const status = error.response.status;
    if (status === 401) {
      forceLogout(); // Token expired
    } else if (status === 400) {
      // Backend validation failed (e.g., email already exists)
      setValidationErrors(error.response.data.errors);
    } else {
      toast.error("Server Error: Please try again later.");
    }
  } else if (error.request) {
    // 2. The request was made but no response was received (e.g., Network disconnected)
    toast.error("Network Error. Check your internet connection.");
  } else {
    // 3. Something happened in setting up the request that triggered an Error
    console.error("Internal Error:", error.message);
  }
}
```

FORM VALIDATION

4. Client-Side Validation

Before sending data to a server, you should validate it on the client side. This provides immediate feedback to the user and prevents unnecessary API calls. While you can write custom `if` statements for every field, the modern standard is to use a schema validation library like **Zod** or **Yup**.

Why Schema Validation? Instead of imperative checks (`if email is empty... if email lacks @...`), you define a declarative "Schema" (a blueprint) of what the data *should* look like. The library then compares your form data against the blueprint and automatically generates detailed error messages if it fails.

Defining the Schema (Zod)

```
// npm install zod
import { z } from 'zod';

// Define the blueprint for our form
const userSchema = z.object({
  username: z.string()
    .min(3, "Username must be 3+ chars"),

  email: z.string()
    .email("Invalid email format"),

  age: z.number()
    .min(18, "Must be at least 18")
});
```

Validating the Data

```
// Inside handleSubmit...
try {
  // 1. Test data against schema
  userSchema.parse(formData);

  // 2. If it passes, submit to API
  await api.post('/users', formData);
} catch (err) {
  // 3. If Zod validation fails, it throws an
  // error
  if (err instanceof z.ZodError) {
    // Formats Zod errors into a usable object
    // e.g. { email: "Invalid email format" }
    const fieldErrors =
      err.flatten().fieldErrors;
    setErrors(fieldErrors);
  }
}
```

Note: Libraries like **React Hook Form integrate seamlessly with Zod/Yup, handling this validation process automatically on every keystroke or blur event.*

DEBUGGING REACT APPS

5. Debugging Strategies

When something goes wrong, blindly placing `console.log()` everywhere is inefficient. React provides specific tools and patterns to track down bugs quickly.

⚙️ REACT DEVTOOLS

The official browser extension is mandatory. It adds two tabs to your browser's inspect panel:

- **Components Tab:** Inspect the live component tree. You can view and dynamically modify the props and state of any component on the fly without refreshing.
- **Profiler Tab:** Record interactions to see exactly which components re-rendered and how long they took, identifying performance bottlenecks.

⚠️ REACT.STRICTMODE

In `'main.jsx'`, your app is wrapped in `<React.StrictMode>`. During development, this intentionally invokes components and effects **twice**.

Why? It helps identify impure functions, missing cleanup logic in `useEffect`, and accidental state mutations. If your app breaks in Strict Mode, you have a hidden bug that needs fixing. *(Note: It only double-renders in development, not production).*

JSON.STRINGIFY TRICK

When debugging complex state objects or arrays, using `'console.log'` can be tricky because the console might evaluate the object late. Instead, render it directly to the screen temporarily:

```
<pre className="bg-gray-800 p-2 text-white text-xs overflow-auto">
  {JSON.stringify(formData, null, 2)}
</pre>
```

This outputs a beautifully formatted JSON block directly in your UI, updating in real-time as you type or change state.

20 TESTING REACT

Manual testing (clicking around the browser to see if things work) is slow, error-prone, and unsustainable for large applications. Automated testing ensures that as you add new features or refactor code, existing functionality does not break.

The Modern Testing Stack

The industry standard for testing React applications relies on two tools working together in harmony: a test runner and a testing utility.

✓ 1. The Test Runner (Jest/Vitest)

Jest (or **Vitest** in modern Vite projects) is the environment where tests run. It finds files ending in `.test.js`, executes the code, and provides the assertion functions (like `expect()`, `toBe()`, `toEqual()`) to verify if the output matches expectations.

⚡ 2. React Testing Library (RTL)

RTL provides tools to render React components into a virtual DOM (provided by `jsdom`) specifically for testing. It provides functions to query elements (like `getByText()`, `getByRole()`) exactly how a user would interact with the screen.

Writing Your First Test

Let's look at the syntax. A test block generally follows the "Arrange, Act, Assert" (AAA) pattern.

```
// Welcome.test.jsx
import { render, screen } from '@testing-library/react';
import Welcome from './Welcome';

// 'test' or 'it' defines a new test case
test('renders the welcome message', () => {
  // 1. ARRANGE: Render the component into the virtual DOM
  render(<Welcome name="Alice" />);

  // 2. ACT: Find the element on the screen
  // We search by text, mimicking a real user looking at the page
  const headingElement = screen.getByText(/Hello, Alice/i); // regex ignore case

  // 3. ASSERT: Check if the element actually exists
  expect(headingElement).toBeInTheDocument();
});
```

TYPES OF TESTING

1. Unit Testing

Unit testing focuses on testing the smallest possible piece of code in isolation. In a React project, this often means testing standard JavaScript utility functions (like math calculations, date formatting, or validation logic) that don't involve rendering UI.

```
// math.test.js (Testing pure JS, no React components)
import { add } from './mathUtils';

test('adds 1 + 2 to equal 3', () => {
  expect(add(1, 2)).toBe(3);
});
```

2. Component Testing

This involves mounting a specific React component and verifying that it renders the correct UI based on given props, and that it responds correctly to user interactions (like clicks or typing).

```
import { render, screen } from '@testing-library/react';
// userEvent simulates realistic browser interactions
import userEvent from '@testing-library/user-event';
import Counter from './Counter';

test('increments counter on button click', async () => {
  // 1. Render the Counter
  render(<Counter />);

  // 2. Verify initial state
  const countDisplay = screen.getByText(/Count: 0/i);
  expect(countDisplay).toBeInTheDocument();

  // 3. Simulate a user clicking the button
  const button = screen.getByRole('button', { name: /Increment/i });
  await userEvent.click(button);

  // 4. Verify the DOM updated correctly
  expect(screen.getByText(/Count: 1/i)).toBeInTheDocument();
});
```

RTL Core Philosophy: "The more your tests resemble the way your software is used, the more confidence they can give you." Notice how we query by what the user *sees* (Text, Role) rather than implementation details (class names or IDs).

MOCKING DEPENDENCIES

Mock Functions (Spies)

When testing a component, we want to isolate it from its dependencies. If a component accepts a function as a prop (like `onSubmit` or `onDelete`), we don't want to actually execute the real function during the test. Instead, we pass a "Mock Function" (a spy) that records if it was called, how many times it was called, and what arguments it received.

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import DeleteButton from './DeleteButton';

test('calls handleDelete with correct ID when clicked', async () => {
  // 1. Create a mock function using jest.fn() or vi.fn() (Vitest)
  const mockHandleDelete = jest.fn();

  // 2. Pass the mock as a prop to the component
  render(<DeleteButton id={99} onDelete={mockHandleDelete} />);

  // 3. Simulate the click interaction
  const button = screen.getByRole('button', { name: /Delete/i });
  await userEvent.click(button);

  // 4. Assert that the mock was executed correctly
  // Verifies it was called exactly once
  expect(mockHandleDelete).toHaveBeenCalledTimes(1);
  // Verifies it was called with the ID 99
  expect(mockHandleDelete).toHaveBeenCalledWith(99);
});
```

Mocking API Calls

Similarly, you should **never make real network requests** during automated testing (it's slow, requires an internet connection, and might alter real database data). Instead, you use tools like MSW (Mock Service Worker) or Jest mocks to intercept the `fetch` or `axios` call and return a fake, predictable JSON response so you can test how your component renders data and handles errors.

INTEGRATION TESTING

3. Integration Testing

Integration testing goes a step further than component testing. Instead of testing a single isolated component, you test how multiple components work together to accomplish a specific user flow.

For example, testing a `<CheckoutFlow />` might involve testing the interactions between a `<Cart />` component, a `<PaymentForm />` component, and the global Redux store state.

The Wrapper Problem

When performing integration testing on components that rely on global Context, Redux, or React Router, the simple `render()` function will fail because the component doesn't have access to the providers.

Solution: You must create a custom render setup that wraps the component under test with the necessary Providers (e.g., `<BrowserRouter>`, `<Provider store={store}>`) before rendering it.

🕒 TESTING PHILOSOPHY SUMMARY

1. DON'T TEST IMPLEMENTATION DETAILS

Don't test if a component's internal state variable changed from `'false'` to `'true'`. Test if the Modal actually appeared on the screen.

2. USE SEMANTIC QUERIES

Prefer `getByRole`, `getByLabelText`, and `getByText` over querying by CSS class names. This also enforces good accessibility practices.

3. ISOLATE WITH MOCKS

Never let a test hit a real backend API or a live database. Intercept network requests and provide mocked JSON responses.

21 AUTHENTICATION

Authentication answers the question "Who are you?", while Authorization answers "What are you allowed to do?". In a modern React Single Page Application (SPA), state is lost upon refreshing. We need a secure, stateless way to remember that a user is logged in as they navigate between routes and make API requests.

1. JWT (JSON Web Tokens)

The industry standard for SPA authentication is **JWT**. A JWT is a secure, encoded string generated by the backend server upon a successful login. The React frontend stores this token and attaches it to the header of every subsequent API request to prove the user's identity.

THE JWT AUTHENTICATION FLOW

- 1 User submits email/password via a React form.
- 2 Backend verifies credentials and signs a unique **JWT string**, returning it to React.
- 3 React saves the JWT locally (e.g., in `localStorage` or an `HttpOnly` cookie) and updates global Auth state.
- 4 For all future requests (like fetching a user profile), React intercepts the request and adds the JWT to the `Authorization: Bearer <token>` header.

Session Management (Where to store the token?)

✗ LocalStorage

```
localStorage.setItem('token', jwt)
```

Easy to implement, but vulnerable to **XSS (Cross-Site Scripting)**. Any malicious JavaScript running on your site can read `LocalStorage` and steal the token.

✓ HttpOnly Cookies

Set by the backend response header.

The token is stored in a special browser cookie that **JavaScript cannot access**, making it immune to XSS. The browser automatically sends it with requests to that domain.

LOGIN, LOGOUT & INTERCEPTORS

2. Login & Logout Actions

Authentication state should be managed globally (using Context API or Redux) so any component can check if a user is logged in.

```
// AuthContext.jsx (Simplified Example)
const login = async (credentials) => {
  try {
    const res = await axios.post('/api/login', credentials);
    const { token, user } = res.data;

    // 1. Save token (If using LocalStorage approach)
    localStorage.setItem('accessToken', token);

    // 2. Update global state
    setAuthUser(user);
    setIsAuthenticated(true);
  } catch (error) {
    throw new Error("Invalid credentials");
  }
};

const logout = () => {
  localStorage.removeItem('accessToken');
  setAuthUser(null);
  setIsAuthenticated(false);
};
```

3. Attaching the Token (Axios Interceptors)

You shouldn't manually add the Authorization header to every single `axios.get()` call in your app. Instead, use an **Axios Interceptor** to automatically intercept outgoing requests and attach the token before it leaves the browser.

```
// api.js
import axios from 'axios';

const api = axios.create({ baseURL: '/api' });

// Intercept REQUESTS before they are sent
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('accessToken');
  if (token) {
    // Inject the token into the headers
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

SECURITY & REFRESH TOKENS

4. Protected Routes

As seen in the Routing chapter, we must prevent unauthenticated users from accessing certain pages. A Protected Route acts as a guard. It checks the global auth state and either renders the requested page (via `<Outlet>`) or redirects them to the login screen.

```
import { Navigate, Outlet, useLocation } from 'react-router';
import { useAuth } from '../context/AuthContext';

const ProtectedRoute = () => {
  const { isAuthenticated, initializing } = useAuth();
  const location = useLocation();

  // Prevent redirecting while React checks localStorage on initial load
  if (initializing) return <LoadingSpinner />;

  // If not logged in, redirect.
  // 'state' saves where they tried to go, so we can redirect them back after login!
  return isAuthenticated ? (
    <Outlet />
  ) : (
    <Navigate to="/login" state={{ from: location }} replace />
  );
};
```

5. The Refresh Token Flow

Access tokens are usually short-lived (e.g., expire in 15 minutes) for security. When they expire, the backend returns a **401 Unauthorized** error. Instead of forcing the user to log in again, we use a long-lived **Refresh Token** to silently request a new Access Token behind the scenes.



Axios Response Interceptor

To implement this, you create an Axios **Response Interceptor**. When a request fails with a 401 error, the interceptor pauses the failure, automatically makes a call to `/api/refresh-token`, updates the saved Access Token, and then silently *retries* the original failed request. If the refresh token is also expired, it finally forces a logout.

PERSISTING STATE & SUMMARY

6. Persisting Auth on Reload

Because React state resets to its initial value (e.g., `user: null`) when the user refreshes the browser, you must check for an existing session when the application mounts.

```
// Inside AuthProvider component
useEffect(() => {
  const initializeAuth = async () => {
    try {
      // 1. Check if token exists in storage
      const token = localStorage.getItem('accessToken');
      if (token) {
        // 2. Verify token is still valid with backend
        const res = await api.get('/auth/me');
        setAuthUser(res.data);
        setIsAuthenticated(true);
      }
    } catch (err) {
      // Token invalid or expired
      localStorage.removeItem('accessToken');
    } finally {
      // 3. Stop the loading spinner so the app can render
      setIsInitializing(false);
    }
  };

  initializeAuth();
}, []);
```

AUTHENTICATION CHECKLIST

1. GLOBAL STATE IS KING

Auth state must be global (Context/Redux) so the Navbar knows to show "Logout" and Protected Routes know whether to allow entry.

2. NEVER TRUST THE CLIENT

Protected Routes only hide the UI. If a user hacks React to bypass a Protected Route, they still won't see data because the backend will reject their API requests if they lack a valid JWT.

3. AUTOMATE WITH INTERCEPTORS

Use Axios Request Interceptors to attach tokens, and Response Interceptors to catch 401s and handle silent token refreshes or forced logouts.

Writing code is only half of a developer's job. Inspecting, debugging, and maintaining the quality of that code requires a robust set of tools. React development relies on a specific "Developer Experience (DX) Stack" to catch bugs early and optimize performance.

1. React DevTools (Browser Extension)

The official **React Developer Tools** extension (available for Chrome, Firefox, and Edge) is mandatory for any React developer. Once installed, it adds two specialized tabs to your browser's Developer Tools panel:

Components and **Profiler**.

● Components Tab	▶ Profiler Tab
Allows you to browse your React component tree exactly as you wrote it in JSX (instead of looking at raw HTML <code>`div`s</code>). • Inspect State & Props: Click any component to instantly see its current state, props, and hooks. • Live Editing: You can double-click state/props values and edit them live to see how the UI reacts without touching your code!	Used to identify performance bottlenecks. You hit "Record", interact with your app, and hit "Stop". • Flamegraph: Shows exactly which components re-rendered during the interaction and <i>why</i> they re-rendered. • Render Times: Highlights components taking too long to render, indicating where you might need <code>React.memo()</code> or <code>useMemo()</code>.

2. Chrome DevTools

While React DevTools is for component logic, standard browser DevTools (F12) are still critical for a React workflow.

Tab	Primary React Use Case
Console	Viewing <code>console.log()</code> outputs, but more importantly, reading React's red error warnings (like missing Keys, unhandled promises, or strict mode warnings).
Network	Essential for API integration. Allows you to see exactly what data was sent in a POST request and the exact JSON payload the server returned.
Application	Where you inspect and manually clear <code>localStorage</code> , <code>sessionStorage</code> , and Cookies (vital when testing Authentication flows).

CODE QUALITY & FORMATTING

3. ESLint (The Linter)

ESLint is a static code analysis tool that scans your JavaScript/React code to find problematic patterns or code that doesn't adhere to specific style guidelines. It acts as an automated spell-checker for logic and syntax errors.

Crucial React Plugin: Modern Vite setups automatically configure `eslint-plugin-react-hooks`. This is the linter rule that yells at you with yellow squiggly lines if you forget to add a variable to a `useEffect` dependency array, or if you try to call a Hook conditionally. **Never ignore these linter warnings!**

```
// Example .eslintrc.cjs Configuration
module.exports = {
  env: { browser: true, es2020: true },
  extends: [
    'eslint:recommended',
    'plugin:react/recommended',
    'plugin:react-hooks/recommended', // ← The lifesaver
  ],
  rules: {
    'react/prop-types': 0, // Disable legacy prop-types requirement
    'no-unused-vars': 'warn' // Warn if a variable is declared but never used
  }
};
```

4. Prettier (The Formatter)

While ESLint focuses on code *quality* and logic, **Prettier** focuses entirely on code *formatting*. It is an "opinionated" code formatter. It takes your messy, inconsistently indented code and reprints it from scratch according to a strict set of rules (tabs vs spaces, single vs double quotes, line width).

```
// Messy Code (Before Prettier)
const App=()=>{
return(<div>
  <h1>Hello</h1><p
className="text">World</p>
</div>)}

```



```
// Clean Code (After Prettier / Save)
const App = () => {
  return (
    <div>
      <h1>Hello</h1>
      <p className="text">World</p>
    </div>
  );
};

```

VS Code Integration: The industry standard workflow is to install both the ESLint and Prettier extensions in VS Code, and set `"editor.formatOnSave": true` in your settings. This way, every time you press `Ctrl+S`, Prettier instantly formats your file, and ESLint instantly highlights any logical errors.

DEVELOPMENT ENVIRONMENT

5. Vite Dev Server & HMR

When you run `npm run dev`, you are launching the **Vite Dev Server**. It is not just serving static files; it is a highly optimized environment specifically designed to enhance developer productivity.



Hot Module Replacement (HMR)

In traditional web development, editing a JavaScript file requires a full page refresh in the browser, which causes you to lose all current application state (e.g., if you were on step 3 of a multi-step form).

React Refresh (via Vite's HMR) solves this. When you save a file, Vite instantly compiles only the specific component you changed and silently swaps it into the live browser **without losing your current state**.



Development vs. Production Build

The code you run in the Vite Dev Server is unoptimized. It contains thousands of React warning messages, unminified variables, and sourcemaps (to help DevTools map errors back to your original source code).

When it's time to deploy, you run `npm run build`. Vite switches to **Rollup** to strip out all development warnings, minify the JavaScript (turning `const userName` into const a`), and bundle the entire application into highly optimized static files ready for a production server.`

</> THE IDEAL REACT DX SETUP

- ✓ **Editor:** VS Code with ESLint and Prettier extensions enabled (Format on Save).
- ✓ **Browser:** Chrome/Firefox with React DevTools installed. Keep the console open to catch React warnings early.
- ✓ **Tooling:** Vite for bootstrapping and instant HMR server.
- ✓ **State Debugging:** Redux DevTools enabled (if using RTK/Zustand) for time-travel debugging of complex state.

23 REACT + BACKEND

React is strictly a UI library—it has no database, no file system access, and no inherent concept of "users". To build a fully functional application, React must communicate with a Backend Server. This communication happens primarily through **REST APIs** using HTTP requests.

1. Custom Backends: Node.js & Express.js

The most natural progression for a React developer is the **MERN Stack** (MongoDB, Express, React, Node). Because both the frontend (React) and backend (Node.js) use JavaScript, you can use the same language, syntax, and tooling across the entire stack. **Express.js** is a minimalist web framework for Node that makes building REST APIs incredibly easy.

server.js (Express Backend)

```
const express = require('express');
const cors = require('cors');
const app = express();

// Middleware to parse JSON & allow React to
// fetch data
app.use(express.json());
app.use(cors());

// REST API Endpoint (GET)
app.get('/api/products', (req, res) => {
  const products = [
    { id: 1, name: 'Laptop' },
    { id: 2, name: 'Phone' }
  ];
  // Send JSON response to React
  res.json(products);
});

app.listen(5000, () => console.log('Server
running'));
```

App.jsx (React Frontend)

```
import { useState, useEffect } from 'react';
import axios from 'axios';

const App = () => {
  const [items, setItems] = useState([]);

  useEffect(() => {
    // Fetch from our custom Express backend
    axios.get('http://localhost:5000/api/products')
      .then(res => setItems(res.data))
      .catch(err => console.error(err));
  }, []);

  return (
    <ul>
      {items.map(i => <li key={i.id}>{i.name}
</li>)}
    </ul>
  );
};
```

What is CORS? Browsers have a strict security policy that prevents a website running on `localhost:5173` (React) from requesting data from a server running on `localhost:5000` (Express). You must use the cors middleware in Express to explicitly allow React to communicate with it.

DATABASES: MONGODB (MERN)

Express can serve hardcoded arrays, but real applications need persistent storage. **MongoDB** is a NoSQL database that stores data in flexible, JSON-like documents (BSON). It pairs perfectly with Node.js because the data format is essentially just standard JavaScript objects.

Mongoose (Object Data Modeling)

To interact with MongoDB from Express, developers use **Mongoose**. It enforces a strict "Schema" on your NoSQL database, ensuring that if you try to save a User without an email, the database rejects it.

```
// models/User.js (Defining the Schema)
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: { type: Number }
});

module.exports = mongoose.model('User', UserSchema);

// server.js (Saving data received from React POST request)
const User = require('./models/User');

app.post('/api/users', async (req, res) => {
  try {
    // 1. req.body contains the JSON sent by React (axios.post)
    const newUser = new User(req.body);

    // 2. .save() writes the document into the MongoDB database
    await newUser.save();

    // 3. Respond back to React with the newly created user (including generated _id)
    res.status(201).json(newUser);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
```

? SQL vs NoSQL for React?

While the MERN stack uses MongoDB (NoSQL), React doesn't care what database you use. The backend completely abstracts the database. You can easily build a PERN stack (PostgreSQL, Express, React, Node) using SQL databases via ORMs like **Prisma** or **Sequelize**, which are highly popular in modern Enterprise development.

BACKEND-AS-A-SERVICE (BAAS)

Building, deploying, and maintaining a custom Node.js backend requires significant effort. For many projects (especially startups and MVPs), developers skip writing custom server code entirely and use a **BaaS (Backend-as-a-Service)**. These platforms provide out-of-the-box databases, authentication, and file storage that React can communicate with directly via an SDK.



Firestore (Google)

The original industry titan. It uses a NoSQL document database called **Firestore**. It is incredibly easy to set up real-time live data syncing (e.g., chat apps) with React.

- NoSQL Database
- Built-in Auth (Google, Email, etc.)
- Cloud Functions for server logic



Supabase

The modern open-source alternative to Firebase. Unlike Firebase, Supabase runs on a powerful **PostgreSQL** relational database, making it vastly superior for complex, structured data.

- SQL (Postgres) Database
- Incredible typed SDK for React
- Row Level Security (RLS)

Example: Fetching Data directly with Supabase

Notice how there is no `fetch` or `axios` call. The SDK handles the REST API logic automatically.

```
import { supabase } from './supabaseClient';

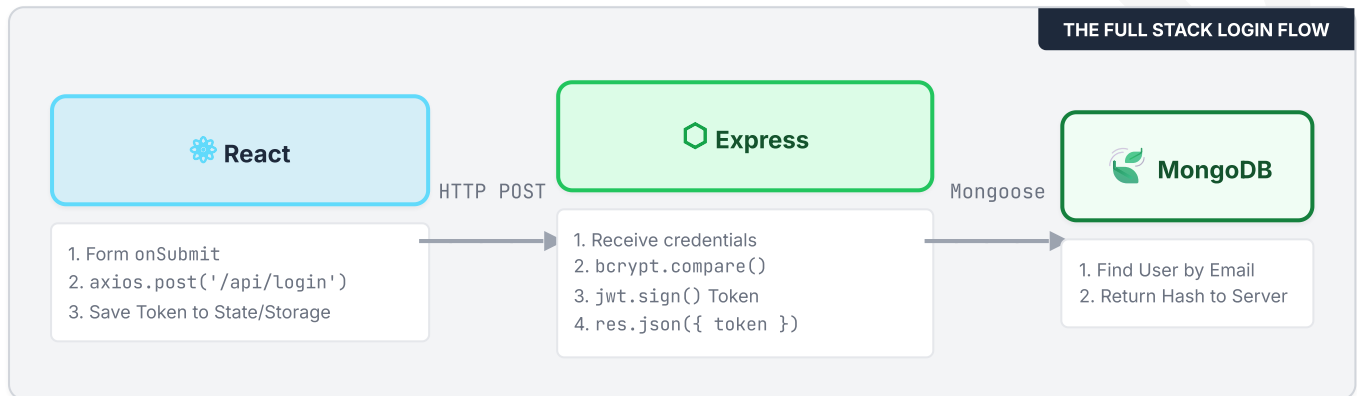
const Products = () => {
  useEffect(() => {
    const fetchProducts = async () => {
      // Translates to: SELECT * FROM products WHERE price > 50
      const { data, error } = await supabase
        .from('products')
        .select('*')
        .gt('price', 50);

      if (error) console.error(error);
      else setProducts(data);
    };
    fetchProducts();
  }, []);
};
```

AUTH APIS & THE FULL PICTURE

Authentication APIs

In Chapter 21, we covered how React handles JWTs on the client side. The Backend API is responsible for actually generating those tokens. A standard auth flow requires endpoints for Registration (hashing passwords with libraries like `bcrypt`), Login (verifying hashes and signing JWTs), and identifying the user from the token.



CHAPTER 23 SUMMARY

- React is just the **View** layer. It cannot directly connect to a database securely without exposing credentials.
- The **MERN Stack** connects React to a custom Node/Express API, which interacts with MongoDB.
- **BaaS (Firebase/Supabase)** provides pre-built backends and SDKs, dramatically speeding up development by bypassing custom API creation.
- Authentication connects both sides: Backend generates a secure token, Frontend stores it and uses it to authorize subsequent API requests.

24 NEXT.JS (FRAMEWORK)

React is a "library" responsible only for rendering UI. To build a production-ready application, you have to make decisions about routing, data fetching, bundling, and SEO. **Next.js** is a comprehensive React framework created by Vercel that makes these decisions for you, providing a complete, optimized full-stack architecture out of the box.

1. Why Next.js? (The SEO Problem)

Standard React apps (like those built with Vite) are **Client-Side Rendered (CSR)**. The server sends an empty HTML file with a `

` and a massive JavaScript bundle. Search engine crawlers (like Googlebot) historically struggle to index CSR pages because they are initially blank. Next.js solves this by rendering the React components on the

Server

first, sending fully formed HTML to the browser.

2. File-Based Routing (App Router)

In standard React, we used `react-router-dom` to map URLs to components manually. Next.js uses a **File-System Based Router**. The folders you create inside the `app` directory automatically become the URL paths of your website.

```
app/
├── layout.tsx
├── page.tsx
├── about/
│   └── page.tsx
├── blog/
│   └── [id]/
│       └── page.tsx
```

app/page.tsx

The `page.tsx` file makes a folder publicly accessible. The `page.tsx` in the root of the `app/` folder represents the homepage URL: `/`.

app/about/page.tsx

By creating an `about` folder and putting a `page.tsx` inside it, Next.js automatically creates the URL: `/about`.

Dynamic Routes: app/blog/[id]/page.tsx

Square brackets indicate a dynamic segment. This file handles `/blog/1`, `/blog/hello-world`, etc. The `id` is passed to the component as a parameter.

SERVER VS. CLIENT COMPONENTS

The most significant paradigm shift in modern Next.js (App Router) is **React Server Components (RSC)**. By default, *every component in Next.js is a Server Component*.

Server Components (Default)

These components are executed entirely on the Node.js server. They never ship JavaScript to the browser, making the bundle size zero. Because they run on the server, they can access databases directly!

```
// Notice: It's an async component!
export default async function Page() {
  // Direct DB fetch on the server
  const users = await db.query('SELECT *
FROM users');

  return <div>{users.length} Users</div>;
}
```

Client Components

These are traditional React components. They are needed whenever you need interactivity (onClick), state (useState), or browser APIs (window). You must explicitly opt-in by adding the "use client" directive.

```
"use client";
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() =>
    setCount(count + 1)}>
    Add
  </button>
}
```

When to use which?

Use Server Components when you need to...	Use Client Components when you need to...
Fetch data directly from a backend/database.	Use interactivity and event listeners (onClick, onChange).
Keep sensitive data (API keys, tokens) secure.	Use State and Lifecycle Hooks (useState, useEffect).
Reduce client-side JavaScript bundle size.	Use browser-only APIs (like window, localStorage).

The "Leaves" Strategy: The best practice in Next.js is to push Client Components to the "leaves" (the very end) of your component tree. Keep your layouts, data fetching, and main page structure as Server Components, and only make small interactive pieces (like a Toggle Button or a Form) Client Components.

DATA FETCHING PARADIGMS & APIS

SSR vs SSG (App Router approach)

Next.js allows you to control exactly *when* a page is rendered: at build time or at request time. In the modern App Router, this is controlled natively by how you configure the standard `fetch` API.

Static Site Generation (SSG)

The page HTML is generated once during `npm run build`. It is then cached on a CDN and served instantly to all users. Ideal for blogs and marketing pages.

```
// Default behavior in Next.js
const res = await
fetch('https://api.com/blog', {
  cache: 'force-cache'
});
```

Server-Side Rendering (SSR)

The page HTML is generated completely from scratch on the server *every time* a user requests it. Necessary for highly dynamic data like a live dashboard.

```
// Opt-out of caching completely
const res = await
fetch('https://api.com/data', {
  cache: 'no-store'
});
```

API Routes (Route Handlers)

Because Next.js runs on a Node.js server, you don't actually need a separate Express backend for simple applications. You can build your REST API directly inside your Next.js project using **Route Handlers**. By creating a `route.ts` file instead of a `page.tsx` file, the folder becomes an API endpoint.

```
// File: app/api/users/route.ts
// This automatically creates the endpoint: GET /api/users
import { NextResponse } from 'next/server';

// The function name MUST match the HTTP method (GET, POST, PUT, DELETE)
export async function GET(request) {
  // You can securely connect to a database here (e.g., Prisma, Mongoose)
  const users = [{ id: 1, name: 'Alice' }];

  return NextResponse.json(users, { status: 200 });
}
```

OPTIMIZATIONS & DEPLOYMENT

Built-in Optimizations

Next.js provides several specialized components that automatically optimize assets to improve Core Web Vitals (like loading speed and visual stability).



<Image /> Component: Replaces the standard `<img>` tag. It automatically compresses images to WebP/AVIF formats, resizes them based on the device screen size, and lazy-loads them only when they enter the viewport, saving massive amounts of bandwidth.



next/font: Automatically optimizes and hosts your Google Fonts (or local fonts) locally at build time, eliminating external network requests and preventing layout shifts (FOUT).



Metadata API: Easily inject dynamic `<title>` and `<meta>` tags per page for flawless SEO and social media sharing cards.

Deployment (Vercel)

While Next.js can be deployed to any Node.js server (like AWS, Render, or DigitalOcean) using `npm run build` and `npm start`, the easiest and most optimized platform is **Vercel** (the creators of Next.js).

When you deploy to Vercel, your static pages (SSG) are automatically distributed across a global **CDN (Content Delivery Network)**. Your Server Components and API Routes are automatically converted into highly scalable **Serverless Functions** or Edge Functions, allowing your app to scale infinitely with zero manual infrastructure management.

CHAPTER 24 SUMMARY

- Next.js transforms React from a client-side library into a powerful full-stack framework.
- Routing is handled automatically by placing folders inside the `app/` directory.
- **React Server Components (RSC)** run on the server, drastically reducing client JS. You must explicitly opt-in to interactivity using `"use client"`.
- Next.js blurs the line between frontend and backend, allowing you to build APIs and connect to databases within the exact same codebase.

25 DEPLOYMENT

Deployment is the process of moving your React application from your local development environment (localhost:5173) to a live server accessible via the internet. Modern cloud platforms have made this process incredibly streamlined, often fully automated via GitHub.

1. The Production Build

You **never** deploy your source code (the src folder with raw JSX). Browsers don't understand JSX natively, and development files are massive. Instead, you create a **Production Build**.

```
// Run this command in your terminal
$ npm run build

// Vite uses Rollup under the hood to:
// 1. Transpile all JSX into plain, highly compatible JS.
// 2. Minify the code (removes spaces, shortens variable names).
// 3. Tree-shake (removes unused library code).
// 4. Output everything into a new folder named 'dist' (or 'build').
```

2. Environment Variables

Applications often require sensitive data like API Keys, database passwords, or different backend URLs for development vs. production. **Environment Variables** allow you to inject these values dynamically without hardcoding them into your source code.

.env (Local File)

```
# 🚫 NEVER commit .env to GitHub!
# Add .env to your .gitignore file.

# SECRET KEY (Not exposed to browser)
DATABASE_PASSWORD=super_secret_123

# PUBLIC KEY (Exposed to browser)
# In Vite, variables MUST start with VITE_
VITE_API_BASE_URL=https://api.prod.com
```

App.jsx

```
// Accessing variables in Vite
const apiUrl =
  import.meta.env.VITE_API_BASE_URL ;

const fetchData = async () => {
  const res = await fetch(`${apiUrl}/users`);
  // ...
};
```

Hosting Environments: Since the .env file is never uploaded to GitHub, you must manually paste your Environment Variables into the settings dashboard of your hosting provider (Vercel, Netlify, Render) before deploying.

FRONTEND HOSTING TITANS

Vercel ▲

The creators of Next.js. Vercel is the industry leader for modern frontend deployment. It offers a global Edge Network, zero-configuration deployments for React/Vite, and seamless GitHub integration.

Best for: Next.js apps, Vite SPAs, Serverless Functions.

Netlify N

The pioneer of the "Jamstack." Highly competitive with Vercel, offering similar automated GitHub deployments. It shines with unique built-in features like Netlify Forms (handles form submissions without a backend) and Identity.

Best for: Static sites, Vite SPAs, drop-in backend features.

The SPA Routing Problem (404 Errors)

If you build a React SPA using React Router, you might notice a catastrophic bug after deploying to Netlify or standard static hosting: Navigating via links works fine, but **if a user refreshes the page on a route like /dashboard, they get a 404 Not Found error!**

Why does this happen?

A static server looks for actual HTML files. When you request /dashboard, the server looks for a dashboard.html file in your build folder. It doesn't exist (because React only outputs index.html).

The Fix: URL Rewrites (Catch-All)

You must configure the hosting server to redirect *all* requests back to index.html. React Router will then take over and render the correct component based on the URL.

```
// Example: Fixing Netlify 404s
// Create a file named exactly '_redirects' in your 'public' folder

/* /index.html 200

# This tells Netlify: "Take ANY request (/*) and instead of looking
# for a file, return the index.html file with a 200 Success status."
```

**Note: Vercel automatically detects Vite/React apps and configures this rewrite for you.*

FULL-STACK & FREE HOSTING

Render (For Full-Stack MERN)

Vercel and Netlify are perfect for frontends, but they are "Serverless" environments. If you built a custom Express.js backend (Chapter 23) that needs to run continuously, listen on a port, or hold websocket connections, you need a different kind of host. **Render** is the modern successor to Heroku.

The MERN Deployment Strategy

- **Frontend:** Host your React app on Vercel.
- **Backend:** Host your Node/Express API on a Render "Web Service".
- **Database:** Host your MongoDB cluster on MongoDB Atlas.

CORS Alert 🚨

When deploying frontend and backend separately, they will have different domains. You **MUST** update your Express CORS settings to explicitly allow requests from your Vercel URL.

GitHub Pages (Portfolios)

If you have a simple static project or a personal portfolio, GitHub provides free hosting directly from your repository via **GitHub Pages**.

```
// Deploying a Vite App to GitHub Pages
// 1. Add your repository name as the 'base' in vite.config.js
import { defineConfig } from 'vite';
export default defineConfig({
  base: '/my-portfolio-repo/',
});

// 2. Push code to GitHub.
// 3. Go to Repo Settings → Pages.
// 4. Set "Source" to GitHub Actions.
// GitHub will detect Vite, build it, and host it automatically!
```

✓ PRE-DEPLOYMENT CHECKLIST

- ✓ Ensure all `console.log` debugging statements are removed.
- ✓ Verify all Environment Variables are configured in the hosting dashboard.
- ✓ Run `npm run build` locally first to ensure there are no hidden compile errors.
- ✓ If using Netlify, ensure the `_redirects` file is in the public folder.

The only way to truly master React is to build. Watching tutorials will only give you the illusion of competence. To escape "Tutorial Hell," you must build projects from scratch, encounter bugs, and read the documentation to fix them. Here is a progressive roadmap of projects.

LEVEL 1: BEGINNER Focus: State, Props, Events, & Basic Hooks**1. Counter & Calculator App**

The "Hello World" of interactive React. Build a simple counter, then upgrade it to a fully functional iOS-style calculator.

Concepts: `useState`, `onClick` events, Conditional Rendering, Grid Layouts.

2. Weather Application

Allow users to search for a city and display the current temperature, humidity, and weather icons using the OpenWeatherMap API.

Concepts: `useEffect`, `Fetch/Axios`, API Keys (Environment Variables), Loading States.

3. Todo Application

A classic CRUD (Create, Read, Update, Delete) app. Users can add tasks, mark them as complete, and delete them.

Concepts: Array State Management, Lists & Keys (`map`), Controlled Forms.

4. Markdown Notes App

Upgrade the Todo app. Allow users to write rich-text notes using Markdown, and save them permanently to the browser.

Concepts: `localStorage` sync, Custom Hooks (`useLocalStorage`), Third-party Markdown parser.

Important: Do not just copy-paste tutorial code for these. Watch a tutorial to understand the architecture, then attempt to build the exact same UI entirely from scratch by memory.

LEVEL 2: INTERMEDIATE

Focus: Routing, Global State, & Complex Data

**5. Movie Database (IMDB Clone)**

Integrate with the TMDB API. Feature a homepage with trending movies, a search bar with auto-complete, and dynamic movie detail pages when clicking on a poster.

Key Skills: React Router (Dynamic URL Params), API Pagination, Debouncing Search Inputs, CSS Grids.

**6. Expense Tracker Dashboard**

Allow users to log income and expenses. Display a visual breakdown of where their money is going using a chart library. Filter transactions by month/year.

Key Skills: Context API / useReducer, Data Aggregation (reduce), Chart.js or Recharts integration.

**7. Serverless Blog Platform**

A multi-page blog. Instead of a custom backend, use a Headless CMS (like Contentful or Sanity) or Firebase to store and fetch articles dynamically.

Key Skills: Headless CMS Integration, Nested Routing, Rich Text Rendering, SEO basics.

LEVEL 3: ADVANCED

Focus: Authentication, Real-time Data, & Scale

Projects at this level are often used as "portfolio centerpieces" to get hired. They require integrating React with robust Backend-as-a-Service (BaaS) platforms like **Firebase** or **Supabase**.

REAL-TIME

8. Chat Application

A WhatsApp clone. Users can log in via Google, see online status, and send real-time messages in global or private chat rooms.

Architecture:

- Firebase Auth (OAuth)
- Firestore (Real-time listeners)
- Zustand / Redux (Global User State)
- Auto-scrolling message lists

ENTERPRISE

9. Admin Dashboard

A data-heavy B2B application. Features complex data tables with sorting/filtering, statistical charts, and a dark/light mode toggle.

Architecture:

- Component Libraries (MUI / Shadcn)
- Data Grids & Pagination
- React Router (Protected Layouts)
- Performance (useMemo for charts)

FULL INTEGRATION

10. E-commerce Website

A complete storefront. Product catalogs with category filters, a global shopping cart, user order history, and a mock (or test) checkout process using the Stripe API.

Frontend Skills:

- Redux Toolkit (Cart Logic)
- Toast Notifications
- Complex Form Validation (Checkout)

Backend / APIs:

- Stripe Payment Element
- Supabase (Database & Auth)
- Framer Motion (Animations)

LEVEL 4: PROFESSIONAL MERN

Focus: Custom Backends, WebSockets, & Next.js

11**Social Media Clone (Instagram/Twitter)**

Custom Node/Express backend. Users can follow others, post images (AWS S3/Cloudinary), like/comment on posts, and view a feed sorted by algorithm. Requires complex relational database querying and infinite scrolling.

12**Food Delivery App (UberEats Clone)**

Multi-role architecture (User, Restaurant Owner, Delivery Driver). Integrates Google Maps API for location tracking, Geocoding for addresses, and real-time order status updates via WebSockets (Socket.io).

13**Learning Management System (LMS)**

THE CAPSTONE

The ultimate test. Build a platform like Udemy using **Next.js**. Teachers can create courses, upload videos (Mux API), and view analytics. Students can buy courses (Stripe), track video progress, and take quizzes. Features Server-Side Rendering, Webhooks, and secure API Routes.

JOURNEY COMPLETE

If you have built, deployed, and understood the projects in this roadmap, you are no longer a beginner. You possess the skills of a **Professional Frontend/MERN Developer**.